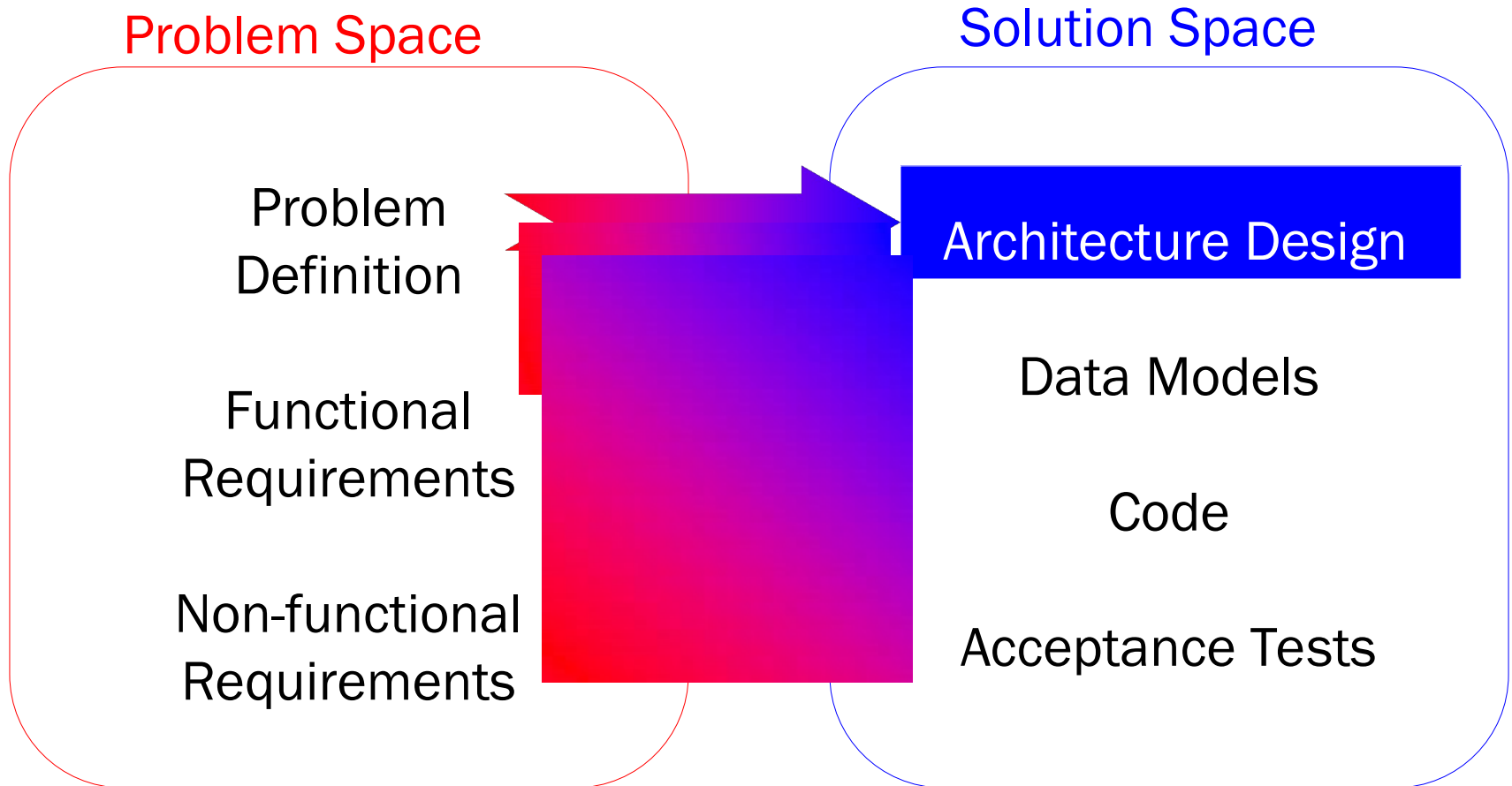

Software Design and Architecture

*By Ashenafi C.
AASTU, 2022*

Chapter Two

Developing Software Architecture

From Requirements to Architecture



Cont'd

- From problem definition to requirements specification
 - » Determine exactly what the customer and user want.
 - » Specifies what the software product is to do.
- From requirements specification to architecture
 - » Decompose software into modules with interfaces.
 - » Specify high-level behavior, interactions, and non-functional properties.
 - » Consider key tradeoffs
 - Schedule vs. Budget
 - Cost vs. Robustness
 - Fault Tolerance vs. Size
 - Security vs. Speed
 - » Maintain a record of design decisions and traceability.
 - » Specifies how the software product is to do its tasks.

Software Architectural Design

- Establishing the **overall structure** of a software system.
- A high-level model of a *system*,
 - » Describes **critical aspects** of the system.
 - » Understandable to many stakeholders.
 - » Allows evaluation of the system's properties before it is built.
 - » Provides well understood tools and techniques for constructing the system from its blueprint.
- Architecture is the **fundamental organization** of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution". [ANSI/IEEE 1471- 400]

Cont'd

- During architectural design an architect should answer;
 - » Which aspects of a software system are ***architecturally relevant***?
 - » How should they be ***represented most effectively*** to enable stakeholders to understand, reason, and communicate about a system before it is built?
 - » What ***tools and techniques*** are useful for implementing an architecture in a manner that preserves its properties?
- **Architecture is:**
 - » All about communication
 - » What 'parts' are there?
 - » How do the 'parts' fit together?
- **Architecture is not:**
 - » About development
 - » About algorithms
 - » About data structures

Architecture-Centric Design

- **Traditional design phase** suggests translating the **requirements into algorithms**, so a programmer can implement them.
- **Architecture-centric design** includes the following considerations:
 - » stakeholder issues.
 - » decision about use of COTS component.
 - » overarching style and structure.
 - » package and primary class structure.
 - » deployment issues.
 - » post implementation/post deployment issues.

Key Architectural Concepts

- **Three *canonical*** building blocks:
 - » Components
 - » Connectors
 - » Configurations
- A ***sub-system*** is a system in its own right whose operation is independent of the services provided by other sub-systems.
- A ***module*** is a system component that provides services to other components but would not normally be considered as a separate system.
- Three fundamental understandings of software architecture:
 - » Every application has an architecture.
 - » Every application has at least one architect.
 - » Architecture is not a phase of development.

Components

- A **software component** is an architectural entity that
 - » encapsulates a subset of the system's functionality and/or data.
 - » restricts access to that subset via an explicitly *defined interface*.
 - » has explicitly defined dependencies on its required execution context.
- Components typically provide application-specific services.
- **Types of components:**
 - » **Computational**: does a computation of some sort. E.g. function, filter.
 - » **Memory**: maintains a collection of persistent data. E.g. data base, file system, symbol table.
 - » **Manager**: contains state + operations. State is retained between invocations of operations. E.g. server.
 - » **Controller**: governs time sequence of events. E.g. control module, scheduler.

Cont'd

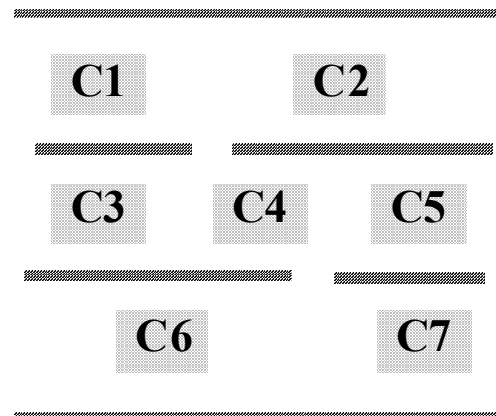
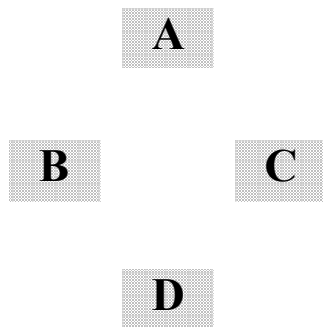
- A *component* is a unit of computation or a data store.
- Components are locus of computation and state.
 - » Clients
 - » Servers
 - » Databases
 - » Filters
 - » Layers
 - » ADTs
- A component may be simple or composite.
 - » Composite components describe a (sub)system.
 - » An architecture consisting of composite components describes a system of systems.

Connectors

- A **software connector** is an architectural building block tasked with ***effecting and regulating interactions*** among components.
- A *connector* is an architectural element that models:
 - » Interactions among components.
 - » Rules that govern those interactions.
- Simple interactions.
 - » Procedure calls
 - » Shared variable access
- Complex and semantically rich interactions.
 - » Client-server protocols
 - » Database access protocols
 - » Asynchronous event multicast
 - » Piped data streams
- Connectors typically provide application-independent interaction facilities.

Configurations/Topologies

- An *architectural configuration* or **topology** is a connected graph of components and connectors that describes **architectural structure**.
 - » Proper connectivity
 - » Concurrent and distributed properties
 - » Adherence to design heuristics and style rules
- Composite components are configurations.



Architectural Drivers

- **Architectural drivers** are the design **forces** that will influence the early design decisions the architects make.
- Architectural drivers are not all of the requirements for a system, but they are an early attempt to identify and capture those requirements, that are **most influential** to the architect making early design decisions.
- Architecturally Significant Requirements.
- (Warning) Architects pay more attention to **qualities** that arise from architecture choices.

Cont'd

- **System requirements:**
 - » **Functional needs** (what the system should do)
 - » **Quality needs** (properties that the system must possess such as availability, performance, security,...)
- **Design constraints**
 - » Development process
 - » Technical constraints
 - » Business constraints
 - » Contractual requirements
 - » Legal obligations
 - » Economic factors

Cont'd

- **MoSCoW Method:**

- » **M - MUST:** Describes a requirement that must be satisfied in the final solution for the solution to be considered a success.
- » **S - SHOULD:** Represents a high-priority item that should be included in the solution if it is possible. This is often a critical requirement but one which can be satisfied in other ways if strictly necessary.
- » **C - COULD:** Describes a requirement which is considered desirable but not necessary. This will be included if time and resources permit.
- » **W - WON'T:** Represents a requirement that stakeholders have agreed will not be implemented in a given release, but may be considered for the future.

Functionality and SA

- It is the ability of the system or application to **satisfy the purpose** for which it was designed.
- It drives the initial **decomposition** of the system.
- It is the **basis** upon which all other quality attributes are specified.
- It is **related to quality attributes** like validity, correctness, interoperability, and security.
- **(Warning)** Functional requirements often get the most focus in a development project. But systems are often redesigned, **not** because of functional requirements.

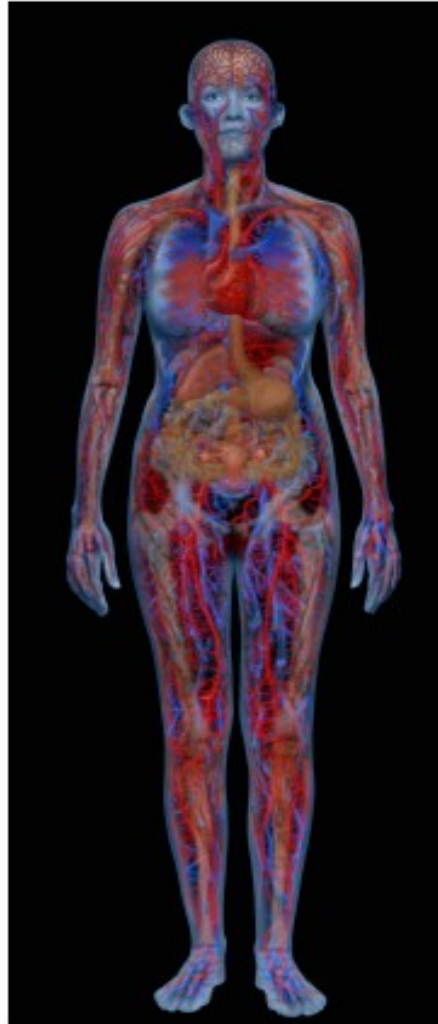
Quality Attributes

- A **quality attribute** is a **measurable** or **testable property** of a system that is used to indicate **how well** the system satisfies the needs of its stakeholders.
- A **quality requirement** is a specification of the **acceptable values** of a quality attribute that must be present in the system.
- Quality attributes should be:
 - » Not subjective
 - » In sufficient detail
 - » Of a value and context (e.g: 180 seconds)

Architectural Structures and Views

- The neurologist, the orthopedist, the hematologist, and the dermatologist all have a ***different view of the structure*** of a ***human body***.
- Ophthalmologists, cardiologists, and podiatrists concentrate on ***subsystems***.
- The kinesiologist and psychiatrist are concerned with different aspects of ***the entire arrangement's behavior***.
- Although these views are ***pictured differently*** and have very ***different properties***, all are inherently related:
- Together they describe the ***architecture*** of the ***human body***.

Physiological Structures



Structures

- Modern systems are more than **complex** enough to make it and difficult to grasp them all at once.
- To communicate meaningfully about an architecture, we must make clear which **structure or structures** we are discussing at the moment—which **view** we are taking of the architecture.
- A **structure** is the **set of elements** itself, as they exist in software or hardware.
 - » For example, a module structure is the set of the system's modules and their organization.

Why Multiple Structures?

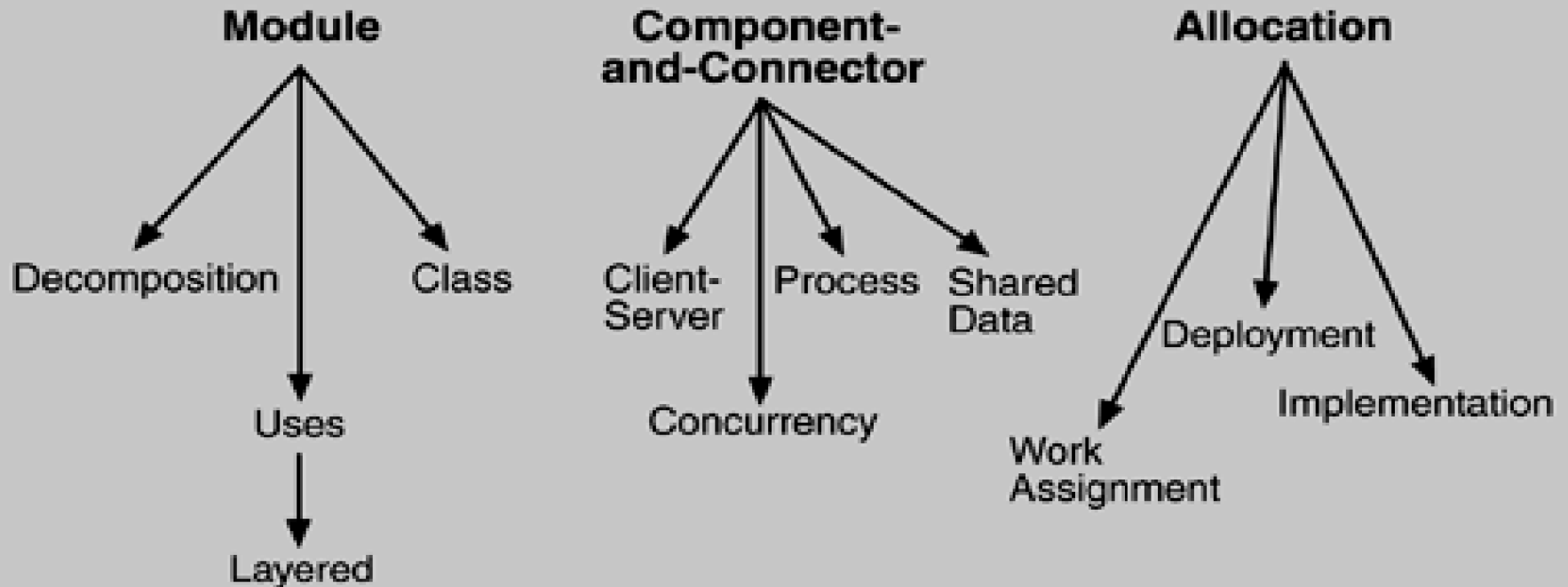
- Software exists in ***multiple component domains*** as a set of ***modules, a set of threads, a set of processes, a set of files***, etc.
- In each component domain a system can form a ***different structure***.
- If all different requirements were supported by ***the same architectural structure*** it would be ***impossible to satisfy them independently***.
- Often system ***requirements may be grouped*** so that requirements in different groups may be addressed by ***different and at least partly independent software structures*** established by partitions of software in different component domains.

Cont'd

- For example:
 - » **Run-time requirements** are addressed by partitioning software into **execution threads** of varying priority (or utility), specifying thread scheduling policies, regulating use of shared resources, etc.
 - » **Portability requirements** are addressed by defining **software layers** and establishing conformance of layers and their interfaces to existing standards.
 - » **Reuse requirements** are addressed by partitioning software into **modules** - substitutable, unit-testable components having well-defined boundaries, predictable interaction with the environment, and minimal, well-specified dependencies on other modules.
 - » **Effective work division** is addressed by partitioning software into **subsystems** that limit the domain expertise necessary for their development and further partitioning the subsystems into separately testable components that implement a meaningful in the product domain function, with minimal interaction with other functions.

Cont'd

- Architectural structures can by large be divided into **three groups**, depending on the broad **nature of the elements** they show.



Cont'd

- **Module structures:** Here the elements are modules, which are *units of implementation*.
- Modules represent a **code-based** way of considering the system.
- They are assigned areas of **functional responsibility**.
- There is less emphasis on how the resulting software manifests itself at **runtime**: give a **static view** of the system.
- Module structures allow us to answer questions such as:
 - » What is the primary functional responsibility assigned to each module?
 - » What other software elements is a module allowed to use? What other software does it actually use?
 - » What modules are related to other modules by generalization or specialization (i.e., inheritance) relationships?

Cont'd

- **Component-and-connector structures:** Here the elements are **runtime components** (which are the principal units of computation) and **connectors** (which are the communication vehicles among components).
- Give a **dynamic view** of the system.
- Component-and-connector structures help answer questions such as:
 - » What are the major executing components and how do they interact?
 - » What are the major shared data stores?
 - » Which parts of the system are replicated?
 - » How does data progress through the system?
 - » What parts of the system can run in parallel?
 - » How can the system's structure change as it executes?

Cont'd

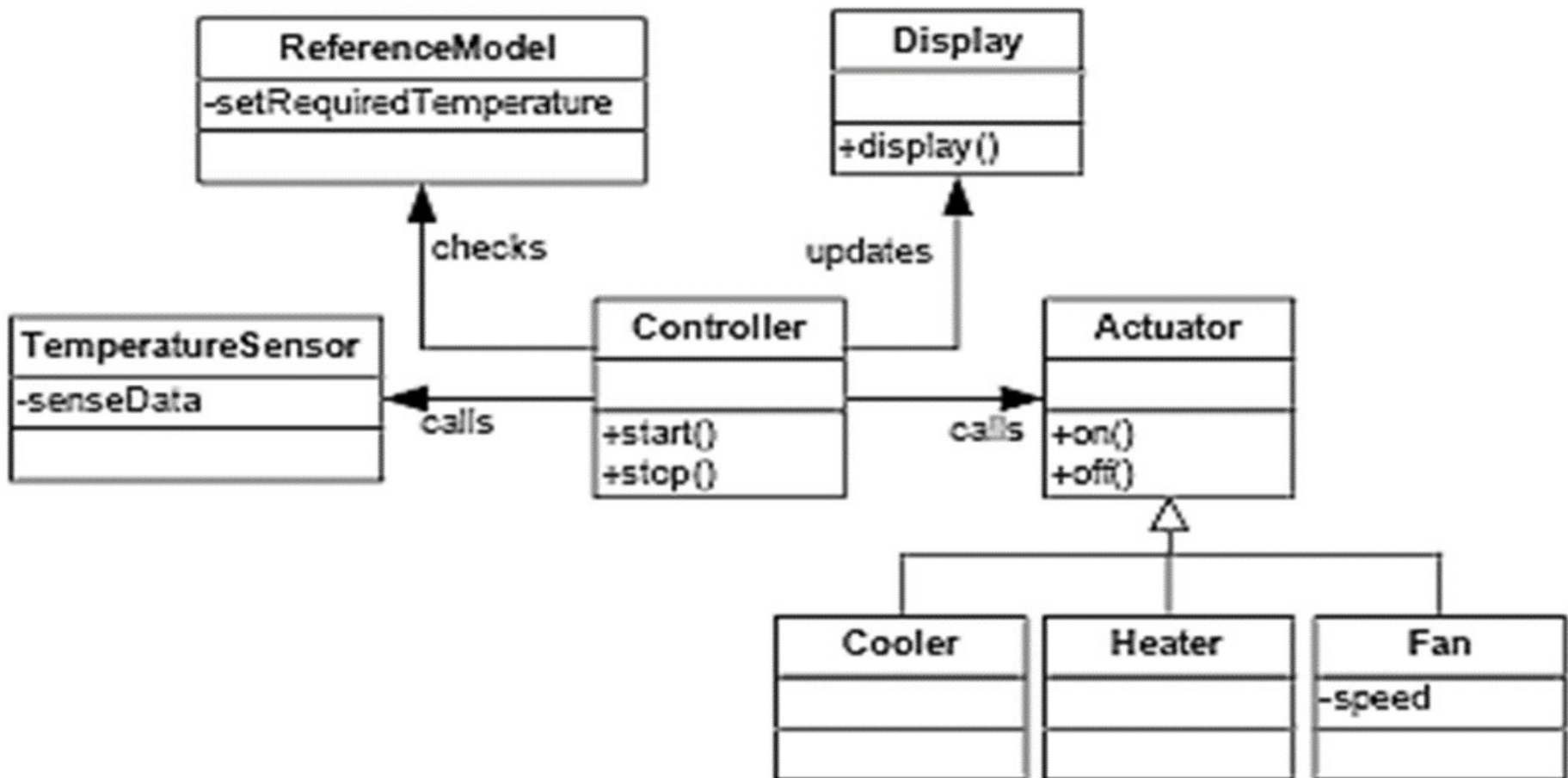
- **Allocation structures:** Allocation structures show the relationship between the software elements and the elements in one or more **external environments** in which the software is created and executed.
- Give a relation the **system and its environment**, such as who is responsible for which part of the system.
- They answer questions such as:
 - » What processor does each software element execute on?
 - » In what files is each element stored during development, testing, and system building?
 - » What is the assignment of software elements to development teams?

Cont'd

- These three structures correspond to the three broad types of **decision** that architectural design involves:
 - » How is the system to be structured as **a set of code units (modules)**?
 - » How is the system to be structured as **a set of elements that have runtime behavior** (components) and **interactions** (connectors)?
 - » How is the system to relate to **non-software structures** in its environment (i.e., CPUs, file systems, networks, development teams, etc.)?

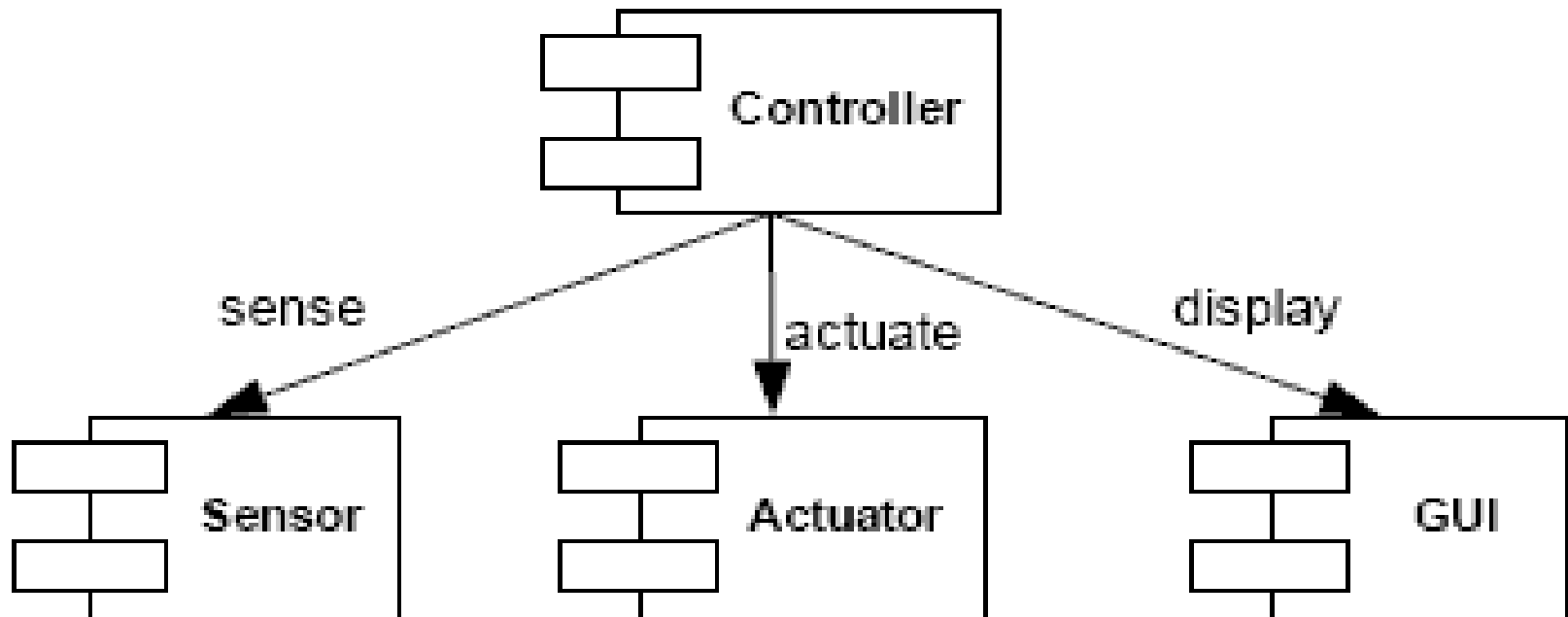
Example: Climate Control System in Vehicles

- Module Structure (Viewpoint)



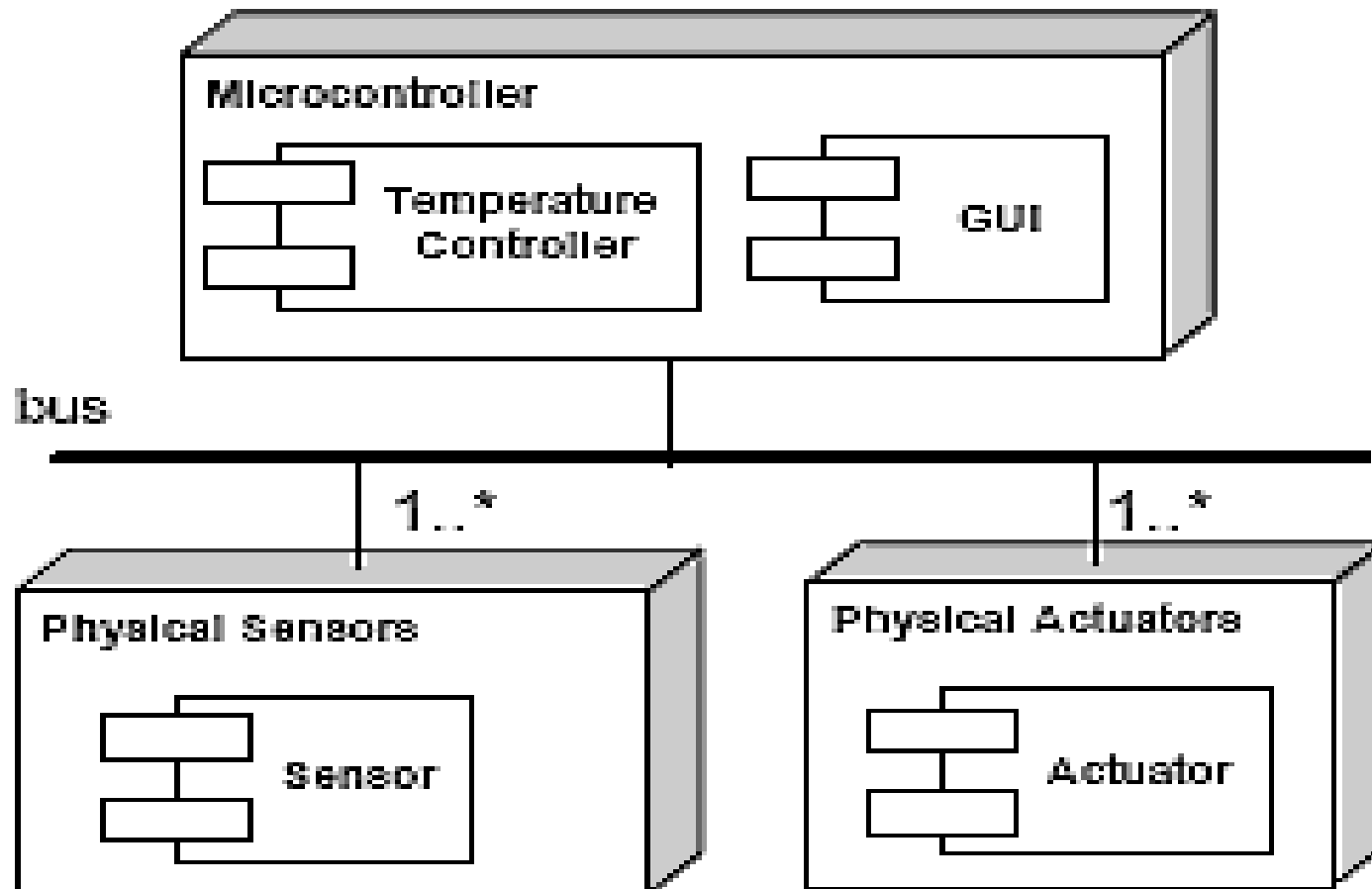
Cont'd

- Component-and-Connector Structure (Viewpoint)



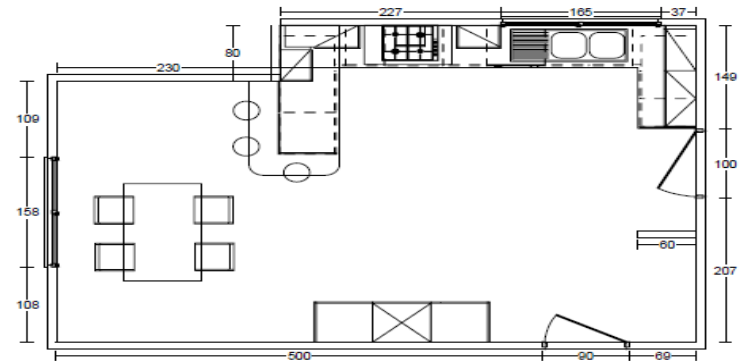
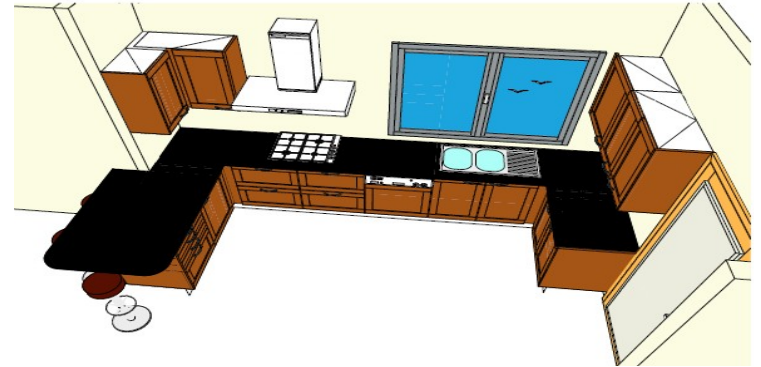
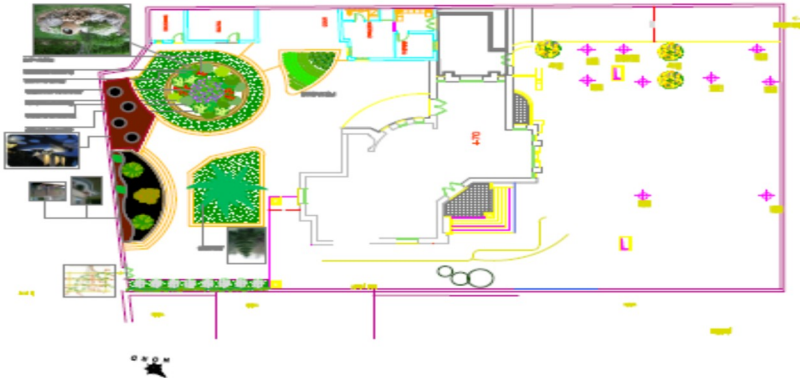
Cont'd

- Allocation Structure (Viewpoint)



Views

- **The analogy with building architecture:** there is no single rendition of a building architecture but many:
 - » the room layouts,
 - » the elevation drawings,
 - » the electrical diagrams,
 - » the plumbing diagrams,
 - » the ventilation diagrams,
 - » the traffic patterns,
 - » the sunlight and passive solar views,
 - » the security system plans, and many others.



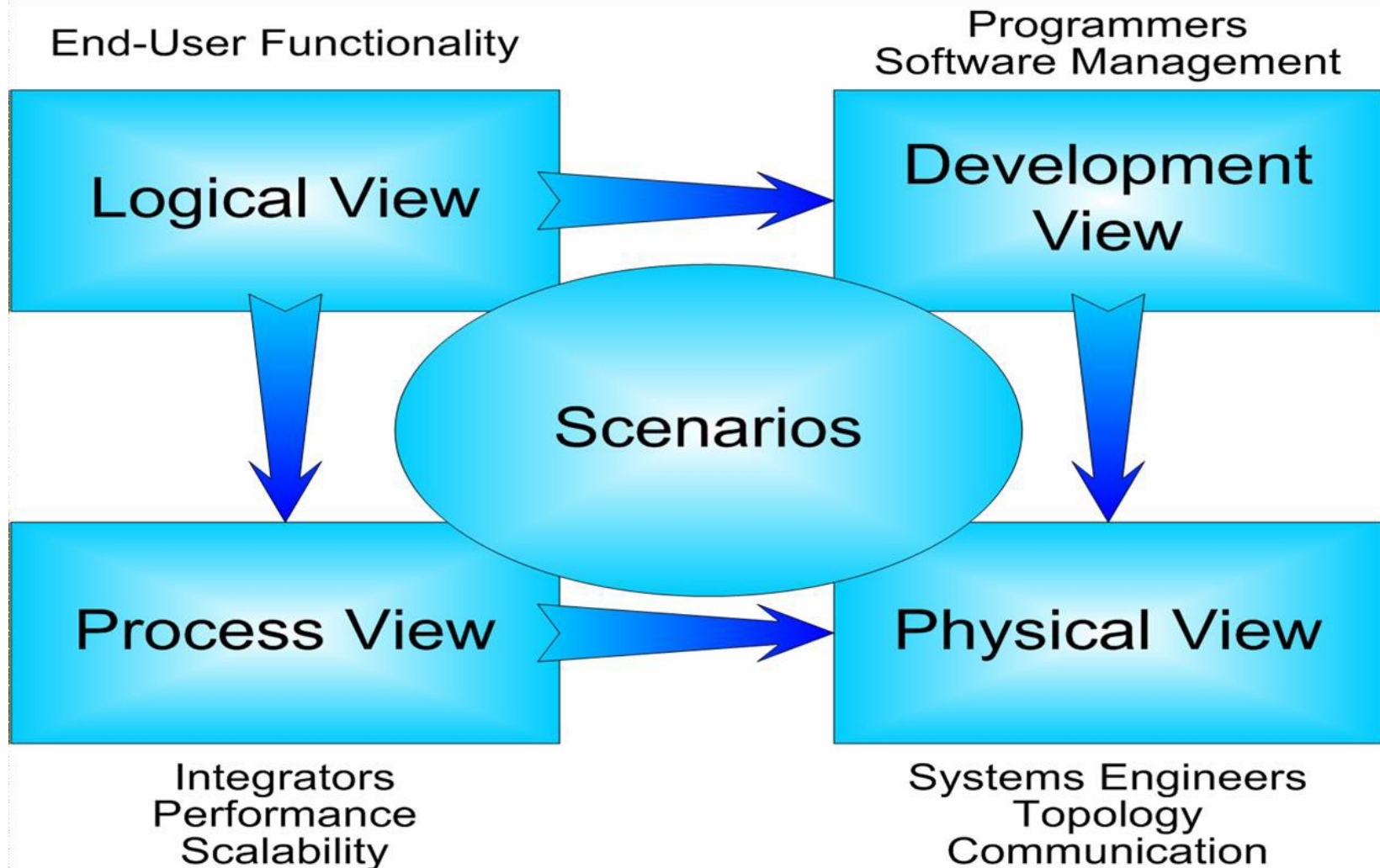
Cont'd

- **View** is a **representation** of a coherent set of architectural elements, as written by and read by system stakeholders.
 - » It consists of a representation of a set of elements and the relations among them.
- **OR View** is a representation of a whole system from the perspective of a ***related set of concerns***.
- **Philippe Kruchten** originally presented the **4+1 View Model** to describe the architecture of software-intensive systems.
- This approach uses ***multiple views*** to separate stakeholders' concerns.
- The 4+1 View Approach is ***widely accepted*** by the software industry to represent application architecture blueprints.

Why Multiple Views?

- Architecture also means ***different things*** to different ***stakeholders***.
- For example,
 - » a ***Network Engineer*** would only be interested in the hardware and network configuration of the system;
 - » a ***Project Manager*** in the key components to be developed and their timelines;
 - » a ***Developer*** in classes that make up a component; and
 - » a ***Tester*** in scenarios.
- So we need multiple view points for distinct stakeholders' needs, showing what is relevant while masking the details that are irrelevant.

4+1 View Model



Cont'd

- **Logical View:** The elements are "**key abstractions**," which are manifested in the object-oriented world as objects or object classes.
- This is a **module view**.
 - » **Focus:** Functional requirements of the system.
 - » **Contents:** Class diagrams, Sequence diagrams, Layer diagrams.
- **Process View:** This view addresses concurrency and distribution of functionality.
- It is a **component-and-connector view**.
 - » **Focus:** Runtime behavior of the system, such as the system processes and communication, concurrency, performance and scalability.
 - » **Contents:** Activity diagrams.

Cont'd

- ***Development (Implementation) View:*** This view shows the organization of software modules, libraries, subsystems, and units of development.
 - » It is an ***allocation view***, mapping software to the development environment.
 - » ***Focus:*** Static organization of the software in its development environment.
 - » ***Contents:*** Component diagram, Package diagrams.
- ***Physical (Deployment) View:*** This view maps other elements onto processing and communication nodes and
 - » It is also an allocation view (which others call the deployment view).
 - » ***Focus:*** System Engineer's perspective, looking at the system topology, deployment and communication.
 - » ***Contents:*** Deployment diagrams.

Cont'd

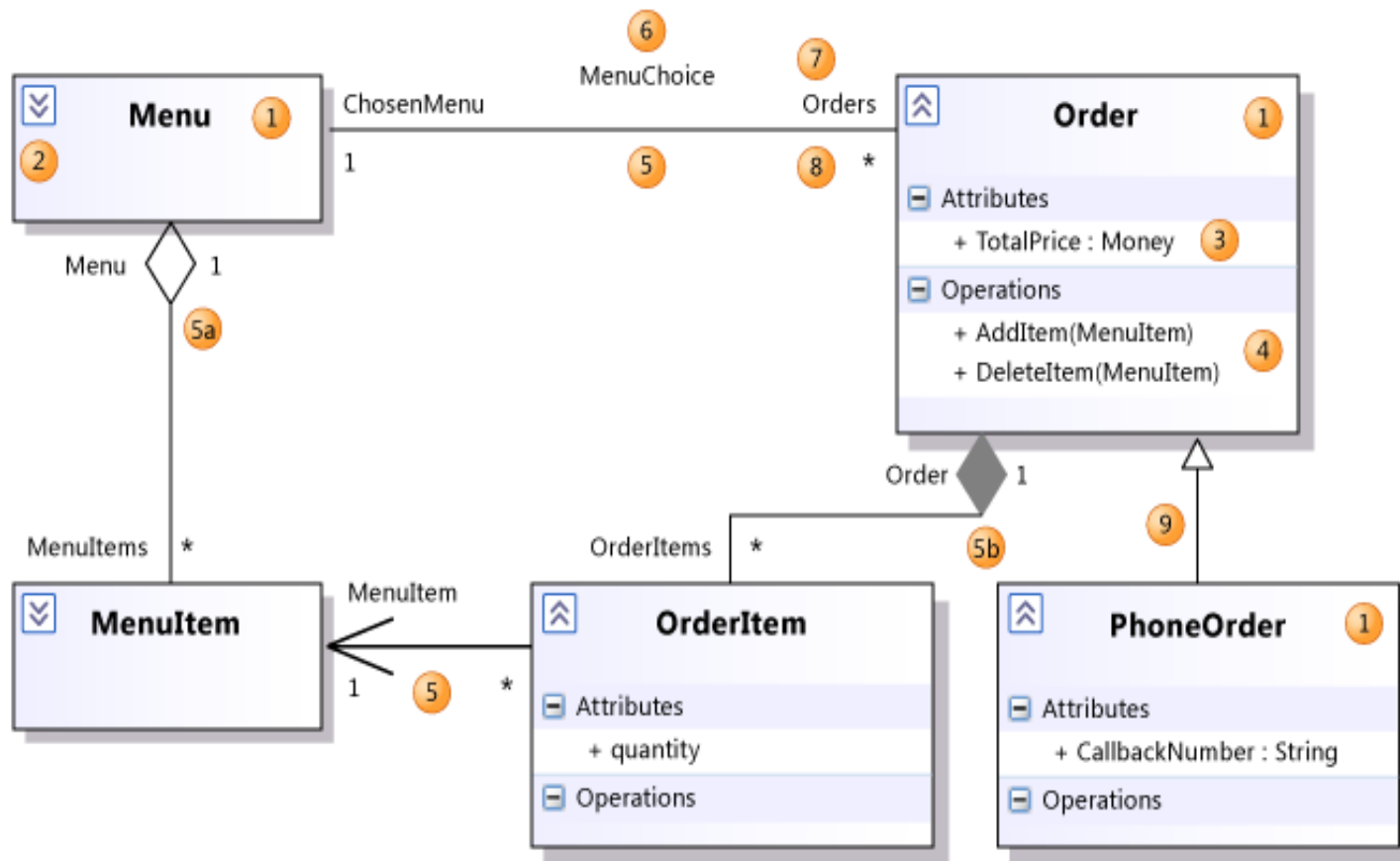
- **Scenarios (Use Case View):** - to validate that the structures were not in conflict with each other and together did in fact describe ***a system meeting its requirements***.
 - » **Focus:** Use cases for illustrating and validating the architecture.
 - » **Contents:** Use case diagrams.
- This view represents the scenarios that tie the four views together, and forms the reason why all the other views exist.
- With all other views in place, this view seems redundant (hence +1).
- However, it represents the architecturally significant requirements in the form of scenarios.
- It also aids to verify that ***all the required scenarios are met***.

Summary: Views

<i>View</i>	<i>Logical</i>	<i>Process</i>	<i>Development</i>	<i>Physical</i>	<i>Scenarios</i>
<i>Components</i>	Class	Task	Module, Subsystem	Node	Step, Scripts
<i>Connectors</i>	association, inheritance, containment	Rendez-vous, Message, broadcast, RPC, etc.	compilation dependency, "with" clause, "include"	Communica- tion medium, LAN, WAN, bus, etc.	
<i>Containers</i>	Class category	Process	Subsystem (library)	Physical subsystem	Web
<i>Stakeholders</i>	End-user	System designer, integrator	Developer, manager	System designer	End-user, developer
<i>Concerns</i>	Functionality	Performance, availability, S/W fault- tolerance, integrity	Organization, reuse, portability, line- of-product	Scalability, performance, av ailability	Understand- ability
<i>Tool support</i>	Rose	UNAS/SALE DADS	Apex, SoDA	UNAS, Openview DADS	Rose

Example: Online Catering Service

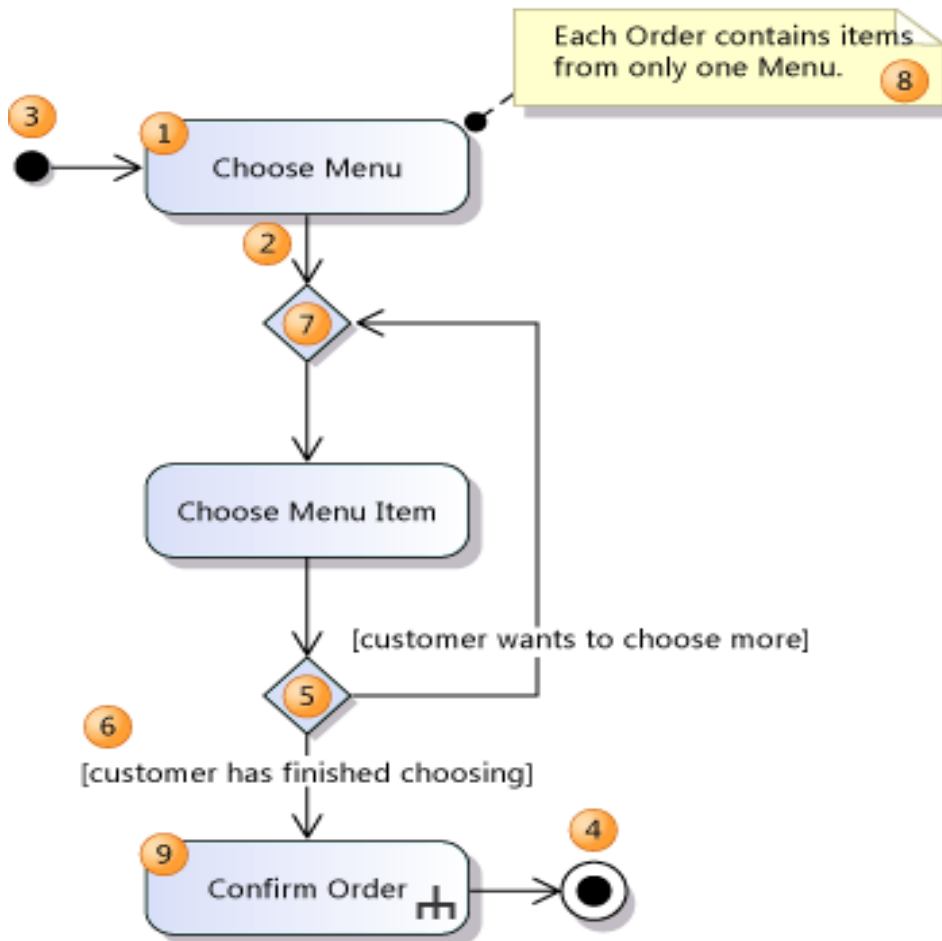
- Logical View: Using Class Diagram



1	Class
3	Attribute
4	Operation
5	Association
5a	Aggregation
5b	Composition
9	Generalization

Cont'd

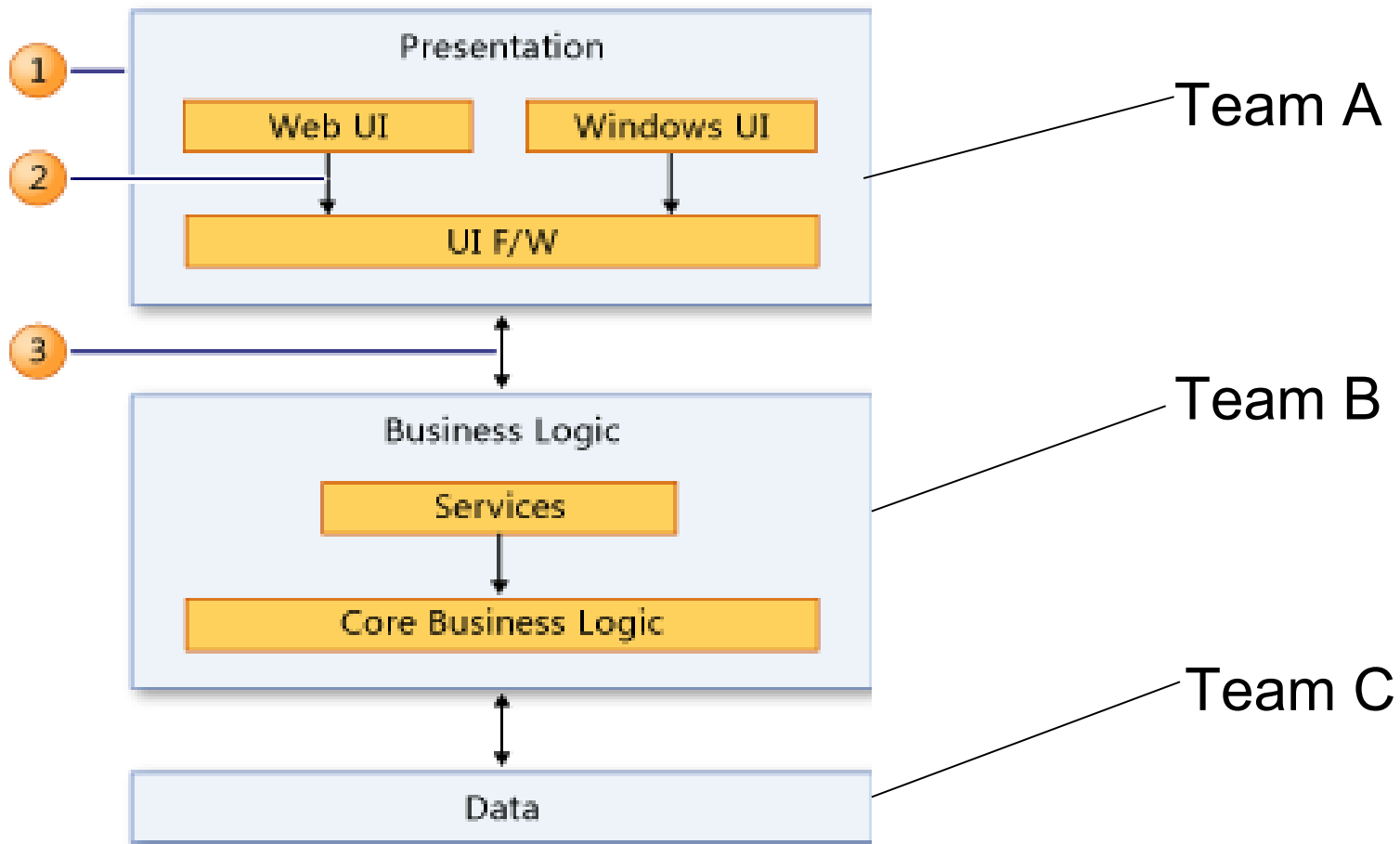
- Process View: Using Activity Diagram



1	Action
2	Control flow
3	Initial node
4	Final node
5	Decision node
7	Merge node

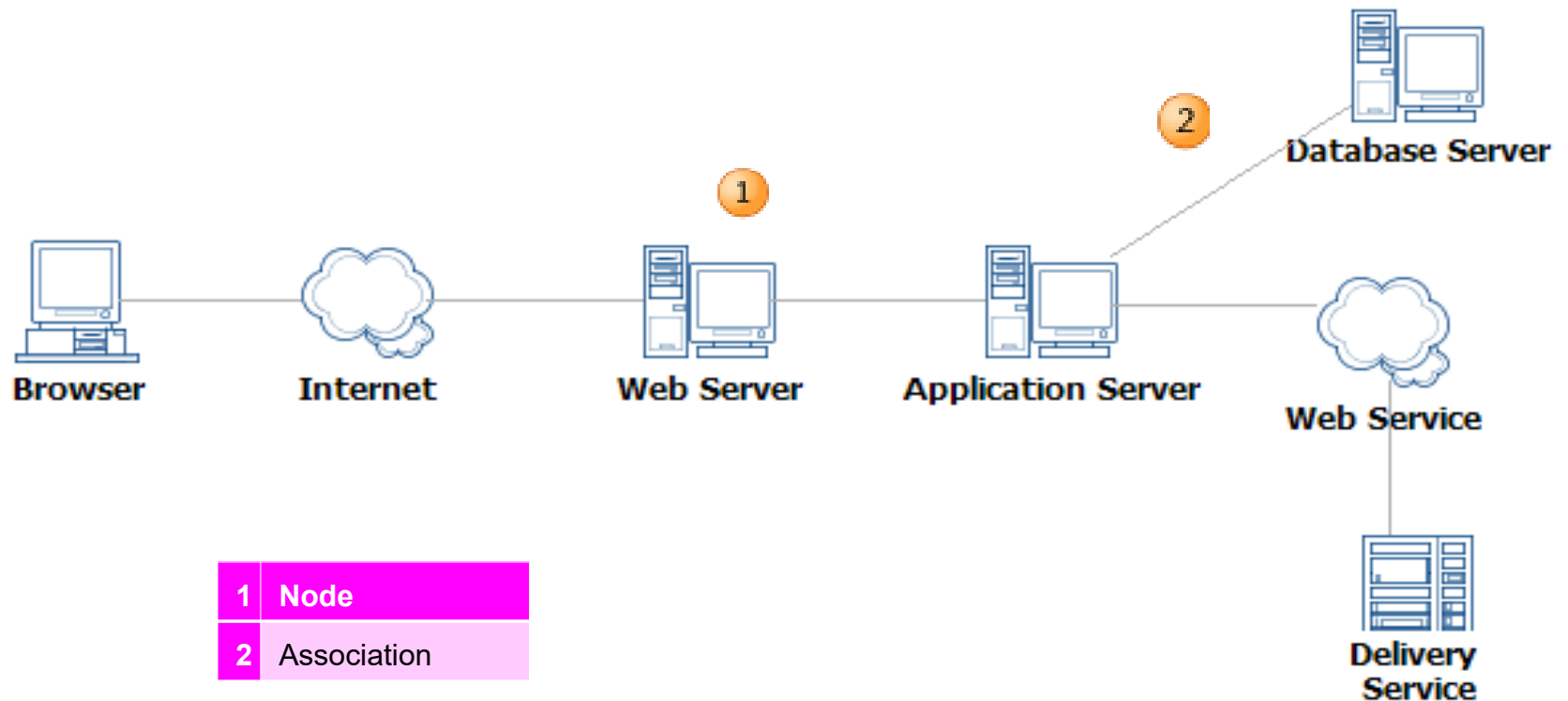
Cont'd

- Development View: Using Layer Diagram



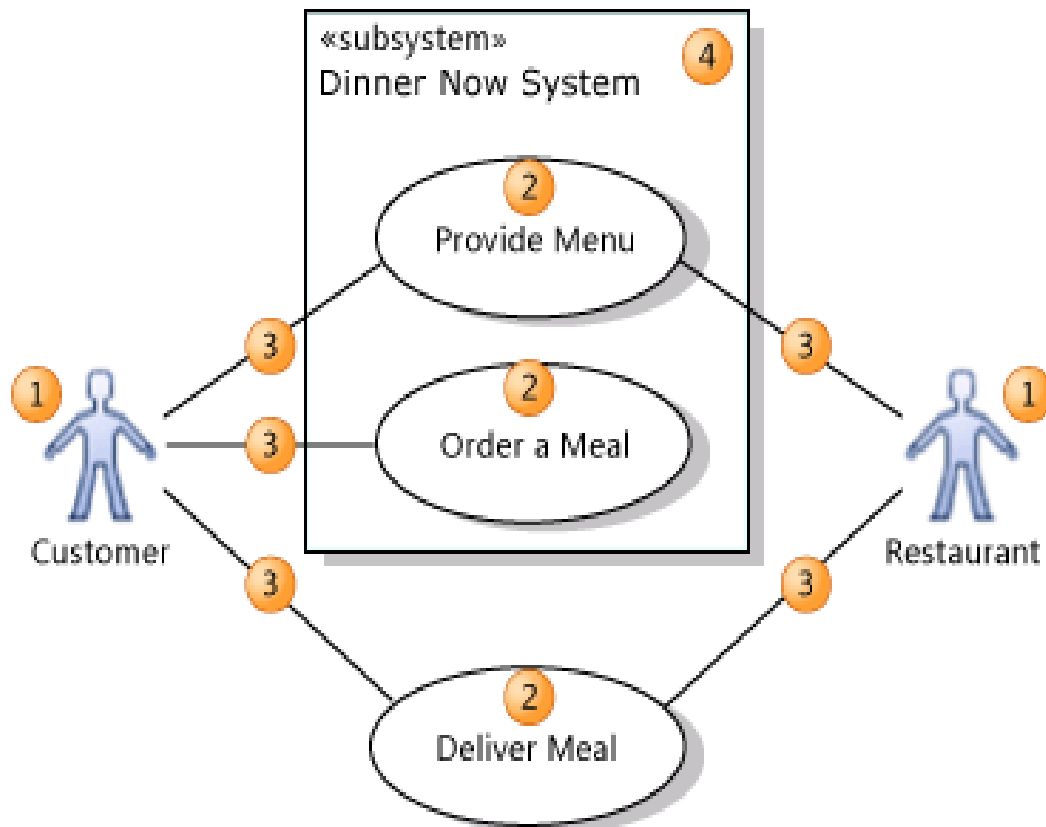
Cont'd

- Physical View: Using Deployment Diagram



Cont'd

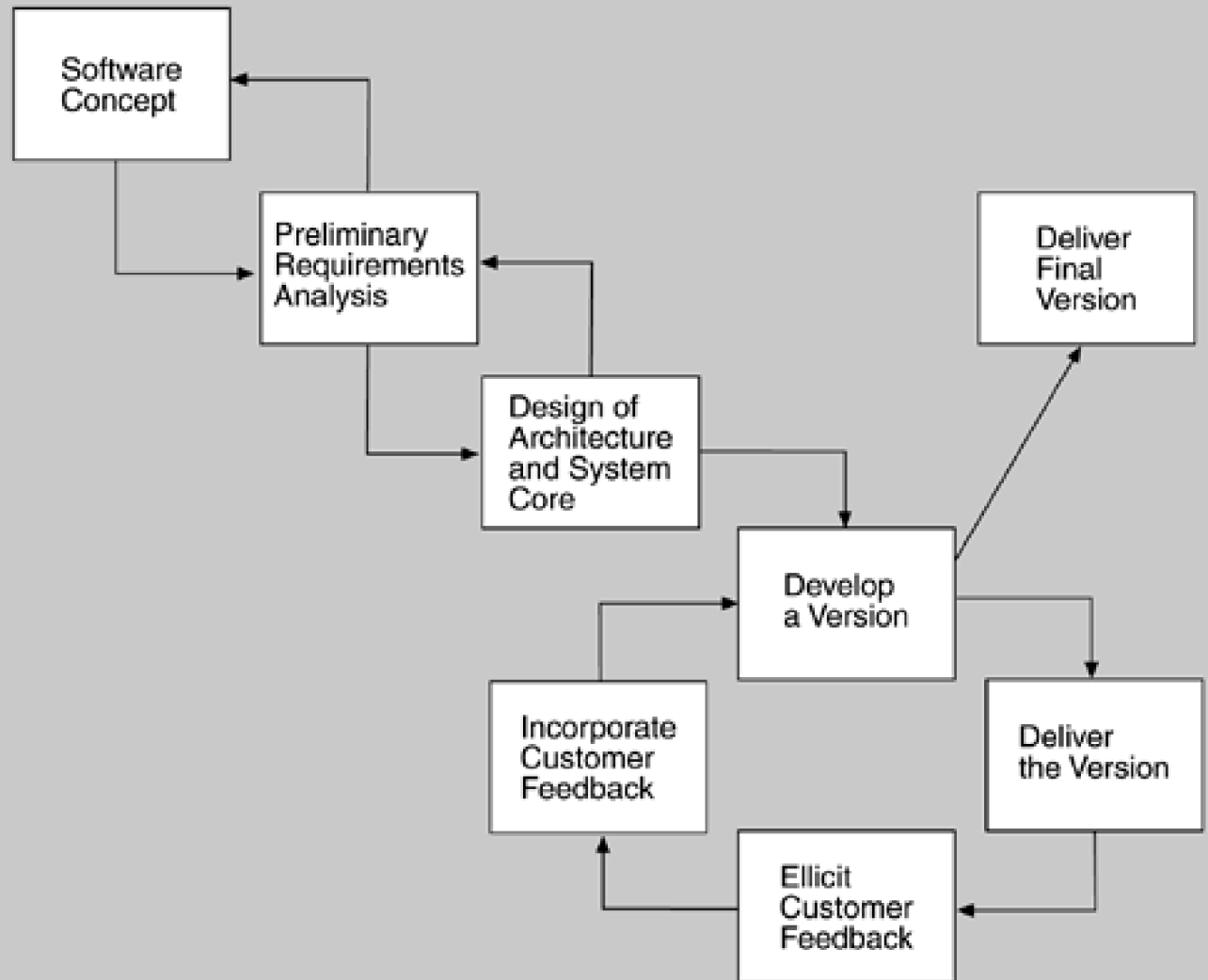
- Scenario: Using Use case Diagram



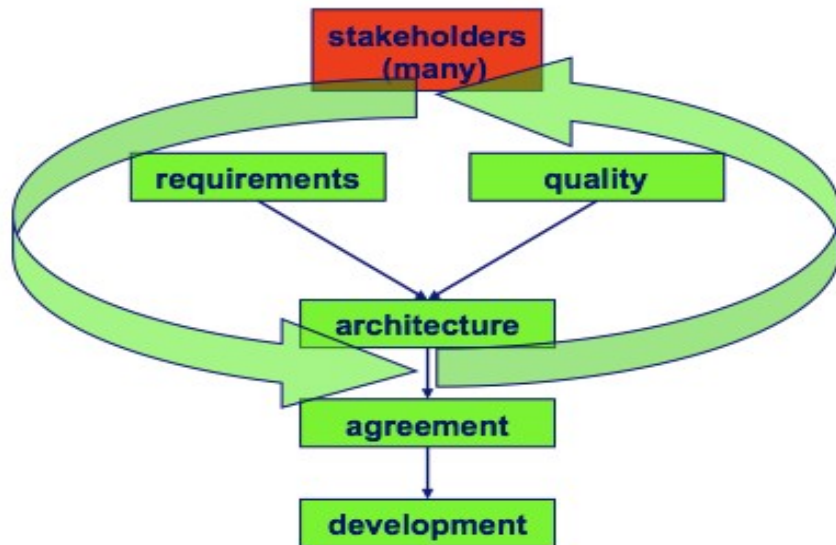
1	Actor
2	Use Case
3	Association
4	Subsystem

The Process of Developing an Architecture

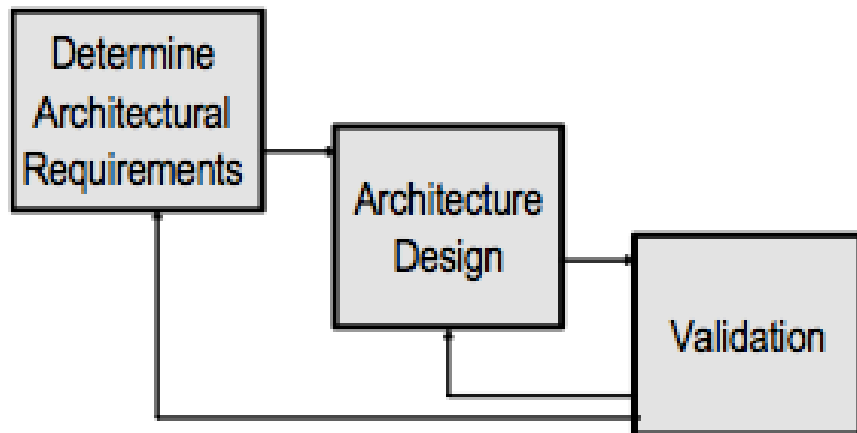
- The **life-cycle model** shows the design of the architecture as iterating with preliminary requirements analysis.
- Clearly, you cannot begin the design until you have some idea of the system requirements.
- On the other hand, it does not take many requirements in order for design to begin.



Architecture in the Life Cycle



- Iteration on both **functional** and **quality** requirements.
- Many *stakeholders* involved.
- Balancing of *functional* and *quality* requirements.



- A simple, **3-step iterative architecture process** that can be used to guide activities during the design.

Steps to Design an Architecture

- **Step 1: Define architecture requirements**
 - » This involves creating a statement or model of the requirements that will drive the architecture design.
- **Step 2: Architecture design**
 - » This involves defining the structure and responsibilities of the components that will comprise the architecture.
- **Step 3: Validation**
 - » This involves “testing” the architecture, typically by walking through the design, against existing requirements and any known or possible future requirements.

1. Identifying Architectural Requirements

- The main sources of architecture requirements are the **functional requirements document** and other documents that reflect various stakeholders' needs.
- Functional requirements tell the ability of the system to do the work for which it was intended.
- These requirements are directly related to the behavior of the system (what it does).
- Some architecture requirements are really **constraints**, for example:

Quality Attribute	Architecture Requirement
Performance	Application performance must provide sub-four second response times for 90% of requests.
Security	All communications must be authenticated and encrypted using certificates.
Resource Management	The server component must run on a low end office-based server with 512MB memory.
Usability	The user interface component must run in an Internet browser to support remote users.

Cont'd

- **Constraints** impose restrictions on the architecture and are (almost always) non-negotiable.
- They limit the range of design choices an architect can make.

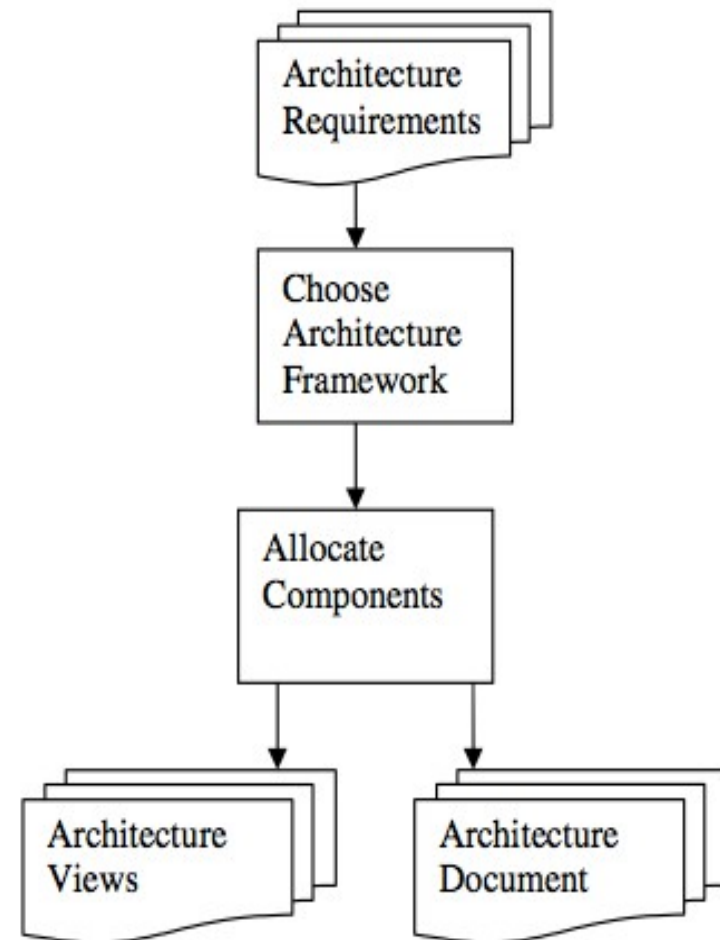
Constraint	Architecture Requirement
Business	The technology must run as a plug-in for MS BizTalk, as we want to sell this to Microsoft.
Development	The system must be written in Java so that we can use existing development staff.
Schedule	The first version of this product must be delivered within six months.

Prioritizing Requirements

- It's necessary to explicitly understand the ***degree of importance*** of architecture requirements and rank them using priorities.
- Initially, it's usually sufficient to allocate each requirement to one of three categories, namely:
 - » **High:** the application must support this requirement. These requirements drive the architecture design
 - » **Medium:** this requirement will need to be supported at some stage, but not necessarily in the first/next release.
 - » **Low:** this is part of the requirements wish list. Solutions that can accommodate these requirements are desired, but they are not the drivers of the design.

2. Architecture Design

- The inputs to the architecture design are the architecture requirements.
- The design stage itself has **two steps**, which are iterative in nature:
 - » Choosing architecture styles
 - » Specifying the individual components that make up the application and allocating them responsibilities.
- The **output** is a set of architecture views that capture the architecture design, and a design document
- The design document explains the **design**, the **key reasons** for some of the major design decisions, and identifies the **risks** involved in taking the design forward.



Choose Architectural Style

- The initial design step involves ***selecting an architecture style***.
 - » The ***architecture style*** that seems likely to satisfy the key requirements should be selected.
 - » For small applications, a single architecture style like n-tier client-server may suffice.
 - » For more complex applications, the design will incorporate one or more known styles.
 - » A pre-requisite, before selecting an architecture style is to understand ***how each of the main architecture styles addresses certain quality attributes***.

Allocate Components

- Once an overall architecture framework has been selected, the next task is to define the major components that will comprise the design.
- The selected architecture style/framework defines the overall communication patterns for the components.
- The activities that follow the selection of the framework include identifying:
 - » major application components, and how they plug into the framework.
 - » the interface or services that each component supports.
 - » responsibilities of the component, stating what it can be relied upon to do when it receives a request.
 - » dependencies between components.
 - » partitions in the architecture that are candidates for distribution over servers in a network.

Cont'd

- The components in the architecture are the **major abstractions** that will exist in the application.
- Some **guidelines** for component design:
 - » Minimize dependencies between components (Look for a **loosely coupled** components).
 - » Design components that encapsulate a highly “cohesive” set of responsibilities.
 - » Use decomposition to structure components hierarchically.
 - » Minimize calls between components, as these can prove costly if the components are distributed.
 - Try to aggregate sequences of calls between components into a single call that can perform the necessary processing in a single request.

3. Architecture Validation

- The aim of the validation phase is ***to increase the confidence of the design team*** that the architecture is ***fit for purpose***.
- The validation has to be achieved within the project constraints of time and budget, as the detailed design and implementation cannot begin until the architecture is agreed.
- Validating an architecture design has some challenges:
 - » Whether it's the architecture for a new application, or an evolution of an existing system, the proposed design is, well, just that – a design.
 - » It can't be executed or tested to see that it fulfills its requirements. It will also likely consist of new components that have to be built and existing applications. All these parts have to be integrated and made to work together.

Cont'd

- There are **two main techniques** that have proved useful.
 - » The first essentially involves **manual testing** of the architecture using test **scenarios**.
 - » The second involves **the construction of a prototype** that creates a simple prototype of the desired application, so that its ability to satisfy requirements can be assessed in more detail through **prototype testing**.
- The aim of both is **to identify potential flaws and weaknesses in the design** so that they can be improved before implementation commences.

The Role of an Architect



Customers



Marketing



Analyst



Architect



Modeler



Developer

Development



Tester -QA



Deployment



IT Operations
Manager



Application
Support
Helpdesk

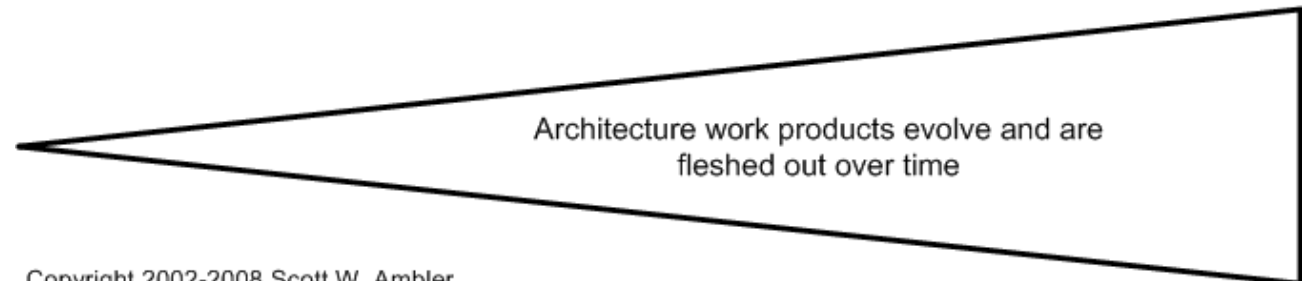
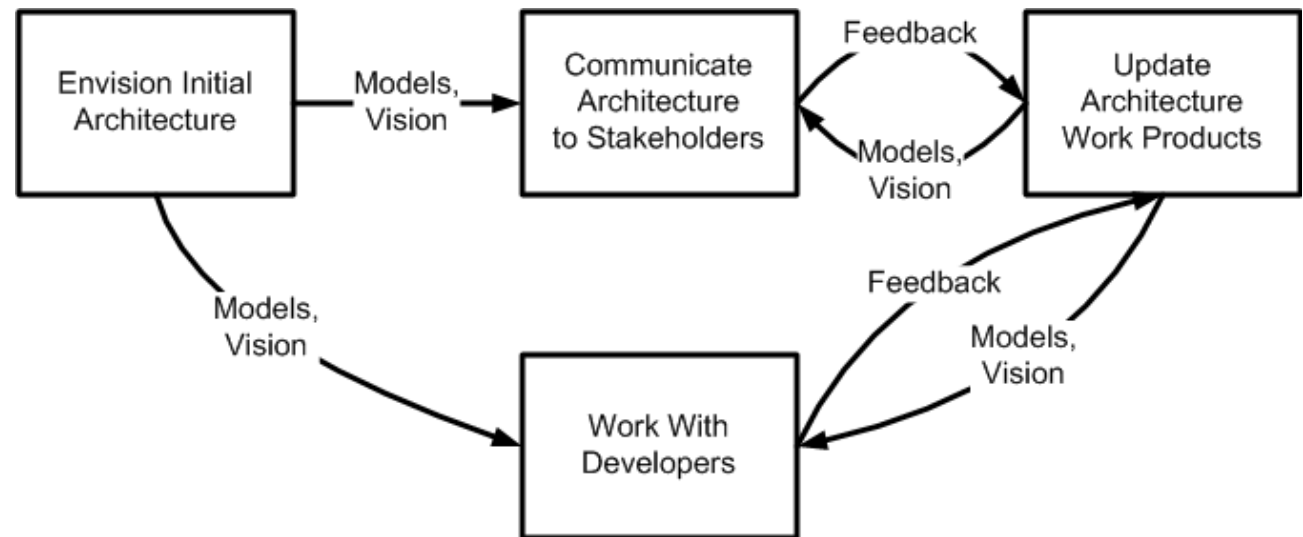
Operations



Cont'd

- Central activities:

- » Design
- » Document
- » Assess
- » Recover
- » Maintain



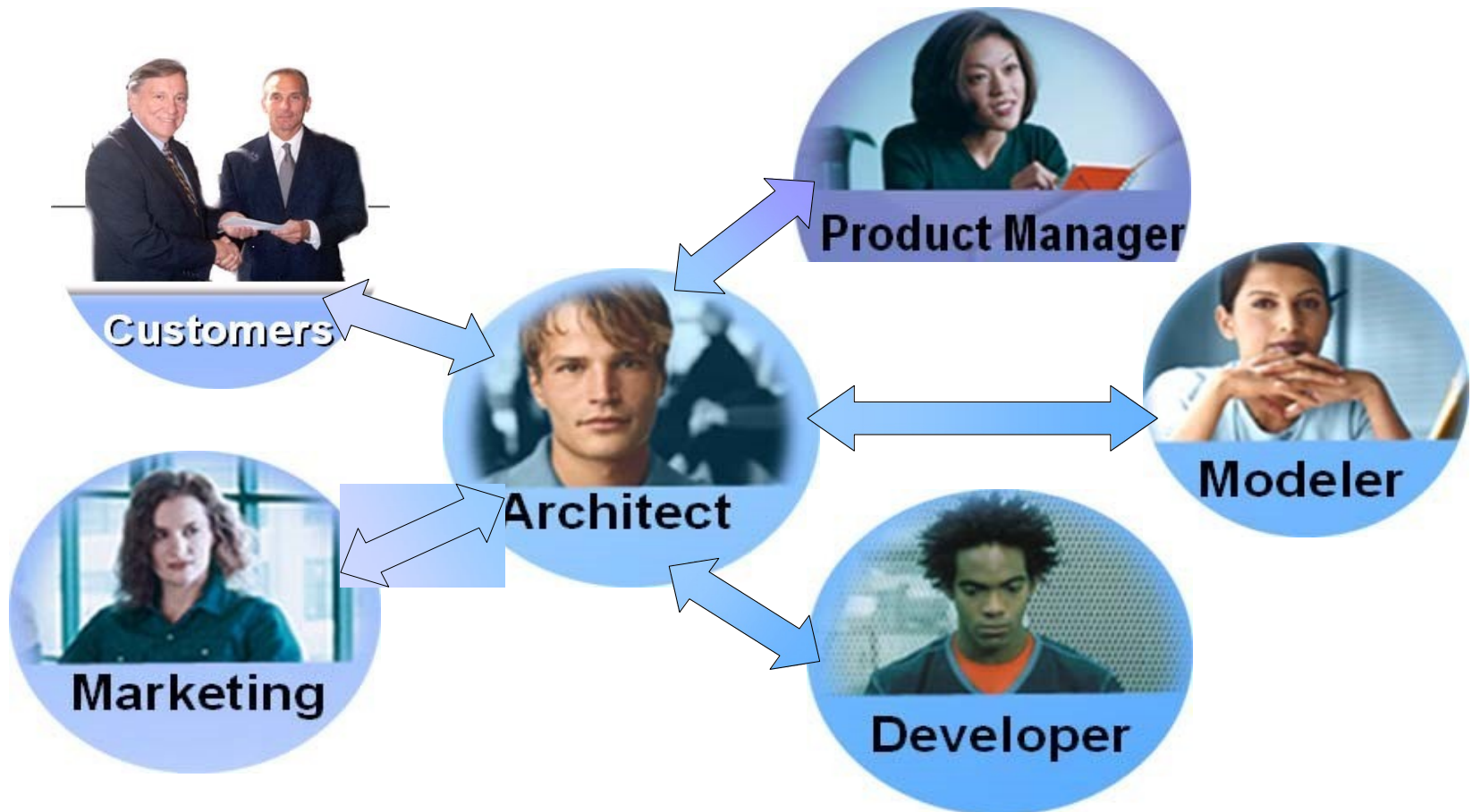
Cont'd

1. Software Engineering Lead

- » Excellent software engineering skills.
- » Lead technical development team by example.
- » Solve the hard problems.
- » Understand impact of decisions.
- » Defend architectural design decisions.
- » Promote good development practices.
- » Plan and manage software releases.

Cont'd

2. Facilitate Communication



Cont'd

3. Technology Expert

- » Know and understand relevant technology.
- » Evaluate and influence the choice of 3rd party frameworks, component and platforms.
- » Track technology development.
- » Know what you don't know.

Cont'd

4. Risk Management

- » Estimate and evaluate risks associated with design options and choices.
- » Document and manage risks, making the whole team (incl. management) aware.
- » Prevent disasters from happening.

Architects' Soft Skills

Technical

- Creative problem solver
- Practical/pragmatic
- Insightful, Investigative
- Tolerant of ambiguity, willing to backtrack, seek multiple solutions
- Good at working at an abstract level

Consulting

- Committed to others' success
- Empathetic, approachable
- An effective change agent, process savvy
- A good mentor, teacher
- Visionary
- Entrepreneurial

Leadership

- Charismatic and credible
- Committed, dedicated, passionate
- Know how to motivate teams

Organizational politics

- Able to see from and sell to multiple viewpoints
- Confident and articulate
- Ambitious and driven
- Patient and not
- Resilient
- Sensitive to where the power is and how it flows in an organization

Traits of the Software Architect

- Gained enough breadth and depth in the relevant domain.
- Strong communication/development/debugging skills.
- Cross domain and technology proficiency and appreciation.
- Knowledge of patterns in domain/technology areas.
- Looking for challenging problems.
- In summary: SUPER ENGINEER.

Quality Attributes in Software Architecture

Architecture and Requirements

System requirements can be categorized as:

- » **Functional requirements** - these requirements state what the system must do, how it must behave or react to run-time stimuli.
- » **Quality attribute requirements** - these requirements annotate (qualify) functional requirements. Qualification might be how fast the function must be performed, how resilient it must be to erroneous input, how easy the function is to learn, etc.
- » **Constraints** - a constraint is a design decision with zero degrees of freedom. That is, it's a design decision that has already been made for you.

Quality Attribute Considerations

- If a functional requirement is "when the user presses the green button the Options dialog appears":
 - » a **performance** QA annotation might describe how quickly the dialog will appear;
 - » an **availability** QA annotation might describe how often this function will fail, and how quickly it will be repaired;
 - » a **usability** QA annotation might describe how easy it is to learn this function.

Quality Attribute Considerations

There are **three problems** with previous discussions of quality attributes:

- 1 The definitions provided for an attribute are not testable. It is meaningless to say that a system will be “modifiable”.
- 1 Endless time is wasted on arguing over which quality a concern belongs to. Is a system failure due to a denial of service attack an aspect of availability, performance, security, or usability?
- 1 Each attribute community has developed its own vocabulary.

Quality Attribute Considerations

- A solution to the **first two** of these problems (untestable definitions and overlapping concerns) is to use ***quality attribute scenarios*** as a means of characterizing quality attributes.
- A solution to the third problem is to provide a discussion of each attribute.
 - » concentrating on its underlying **concerns** to illustrate the concepts that are fundamental to that attribute community.

Specifying Quality Attribute Requirements

- We use a common form to specify all quality attribute requirements as scenarios.
- Our representation of quality attribute scenarios has these parts:
 1. Stimulus
 2. Stimulus source
 3. Response
 4. Response measure
 5. Environment
 6. Artifact

Specifying Quality Attribute Requirements

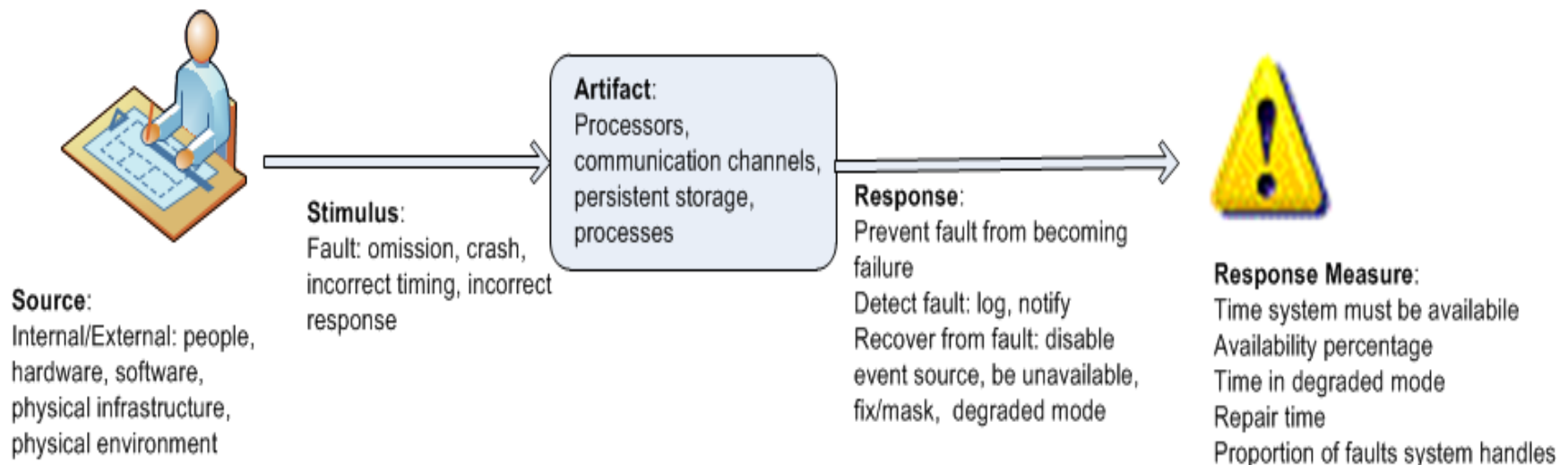
1. **Source of stimulus** - this is some **entity** (a human, a computer system, or any other actuator) that generated the stimulus.
2. **Stimulus** - the stimulus is a **condition** that requires a response when it arrives at a system.
3. **Environment** - the stimulus occurs under certain conditions. The system may be in an overload condition or in normal operation, or some other relevant state. For many systems, “normal” operation can refer to one of a number of modes.
4. **Artifact** - some artifact is stimulated. This may be a collection of systems, the whole system, or some piece or pieces of it.
5. **Response** - the response is the activity undertaken as the result of the arrival of the stimulus.
6. **Response measure** - when the response occurs, it should be measurable in some fashion so that the requirement can be tested.

Specifying Quality Attribute Requirements

- We distinguish
 - » **general** quality attribute scenarios (“general scenarios”) - those that are system independent and can, potentially, pertain to any system from *concrete* quality attribute scenarios (concrete scenarios).
 - » **concrete** quality attribute scenarios (“concrete scenarios”) - those that are specific to the particular system under consideration.

Specifying Quality Attribute Requirements

Example **general** scenario for **availability**:



Achieving Quality Attributes Through Tactics

- There are a collection of primitive design techniques that an architect can use to achieve a quality attribute response.
- We call these architectural design primitives ***tactics***.
- **Tactics**, like **design patterns**, are techniques that architects have been using for years.
- We do not *invent* tactics, we simply capture what architects do in practice.

Achieving Quality Attributes Through Tactics

Why do we do this? There are **three** reasons:

- » Design patterns are complex; they are a bundle of design decisions. But patterns are often difficult to apply as is; **architects need to modify and adapt them**. By understanding tactics, an architect can assess the options for augmenting an existing pattern to achieve a quality attribute goal.
- » If no pattern exists to realize the architect's design goal, tactics allow the architect to construct a design fragment from "first principles".
- » By cataloguing tactics, we make design more systematic. You frequently will have a **choice of multiple tactics** to improve a particular quality attribute. **The choice of which tactic to use depends on factors such as tradeoffs among other quality attributes and the cost to implement.**

Guiding Quality Design Decisions

- Architecture design is a systematic approach to making design decisions.
- We categorize the **design decisions** that an architect needs to make as follows:
 1. Allocation of responsibilities
 2. Coordination model
 3. Data model
 4. Management of resources
 5. Mapping among architectural elements
 6. Binding time decisions
 7. Choice of technology

Allocation of Responsibilities

- Decisions involving allocation of responsibilities include:
 - » identifying the **important responsibilities** including basic system functions, architectural infrastructure, and satisfaction of quality attributes.
 - » determining how these responsibilities are allocated to non-runtime and runtime elements (namely, modules, components, and connectors).

Coordination Model

- Decisions about the coordination model include:
 - » identify the elements of the system that must coordinate, or are prohibited from coordinating.
 - » determining the properties of the coordination, such as timeliness, currency, completeness, correctness, and consistency.
 - » choosing the communication mechanisms that realize those properties.
 - Important properties of the communication mechanisms include stateful vs. stateless, synchronous vs. asynchronous, guaranteed vs. non-guaranteed delivery, and performance-related properties such as throughput and latency.

Data Model

- Decisions about the data model include:
 - » choosing the major data abstractions, their operations, and their properties. This includes determining how the data items are created, initialized, accessed, persisted, manipulated, translated, and destroyed.
 - » metadata needed for consistent interpretation of the data.
 - » organization of the data. This includes determining whether the data is going to be kept in a relational data base, a collection of objects or both.

Management of Resources

- Decisions for management of resources include:
 - » identifying the resources that must be managed and determining the limits for each.
 - » determining which system element(s) manage each resource.
 - » determining how resources are shared and the arbitration strategies employed when there is contention.
 - » determining the impact of saturation on different resources.

Mapping Among Architectural Elements

- Useful mappings include:
 - » the mapping of modules and runtime elements to each other—that is, the runtime elements that are created from each module; the modules that contain the code for each runtime element.
 - » the assignment of runtime elements to processors.
 - » the assignment of items in the data model to data stores.
 - » the mapping of modules and runtime elements to units of delivery.

Binding Time

- The decisions in the other categories have an associated binding time decision.
- Examples of such binding time decisions include:
 - » For allocation of responsibilities you can have build-time selection of modules via a parameterized build script.
 - » For choice of coordination model you can design run-time negotiation of protocols.
 - » For resource management you can design a system to accept new peripheral devices plugged in at run-time.
 - » For choice of technology you can build an app-store for a smart phone that automatically downloads the appropriate version of the app.

Choice of Technology

- Choice of technology decisions involve:
 - » deciding which technologies are available to realize the decisions made in the other categories.
 - » determining whether the tools to support this technology (IDEs, simulators, testing tools, etc.) are adequate.
 - » determining the extent of internal familiarity and external support for the technology (such as courses, tutorials, examples, availability of contractors).
 - » determining the side effects of choosing a technology such as a required coordination model or constrained resource management opportunities.
 - » determining whether a new technology is compatible with the existing technology stack.

Availability

- **Availability** refers to a property of software that it is there and ready to carry out its task when you need it to be.
- This is a broad perspective and encompasses what is normally called **reliability**.
- Availability builds on reliability by adding the notion of recovery (repair).
- Fundamentally, availability is about minimizing service outage time by mitigating faults.

Availability General Scenario

Portion of Scenario	Possible Values
Source	Internal/external: people, hardware, software, physical infrastructure, physical environment
Stimulus Artifact	Fault: omission, crash, incorrect timing, incorrect response System's processors, communication channels, persistent storage, processes
Environment	Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation
Response	Prevent the fault from becoming a failure Detect the fault: • log the fault • notify appropriate entities (people or systems) Recover from the fault • disable source of events causing the fault • be temporarily unavailable while repair is being effected • fix or mask the fault/failure or contain the damage it causes • operate in a degraded mode while repair is being effected
Response Measure	Time or time interval when the system must be available Availability percentage (e.g. 99.999%) Time to detect the fault Time to repair the fault Time or time interval in which system can be in degraded mode Proportion (e.g., 99%) or rate (e.g., up to 100 per second) of a certain class of faults that the system prevents, or handles without failing

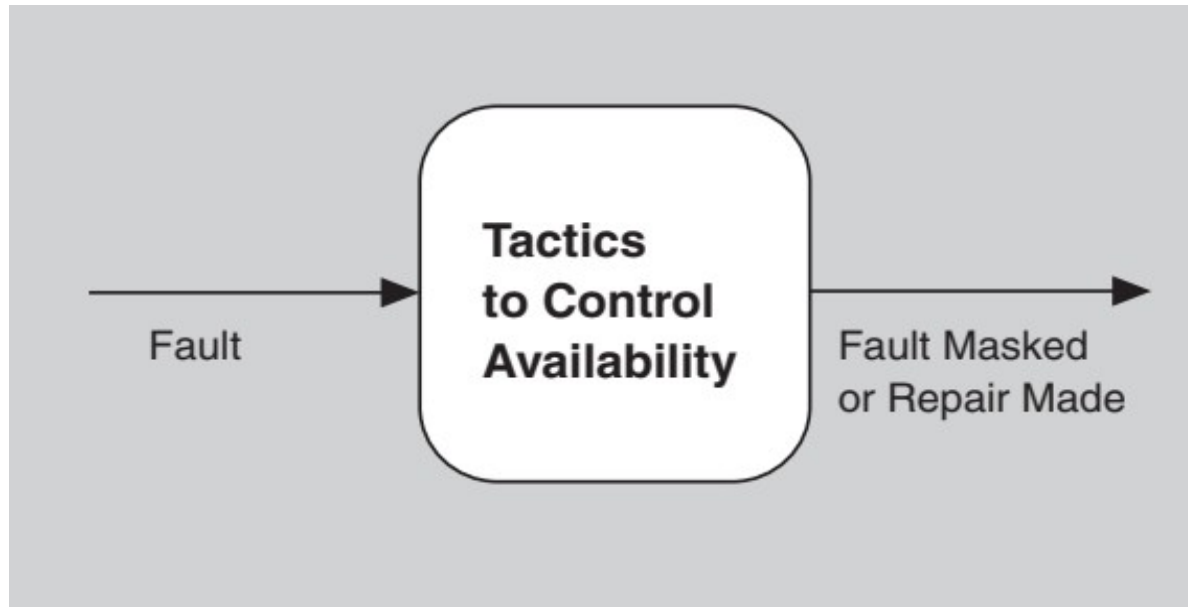
Sample Concrete Availability Scenario

- The **heartbeat monitor** determines that the server is nonresponsive during normal operations.
- The system **informs the operator** and continues to operate with no downtime.

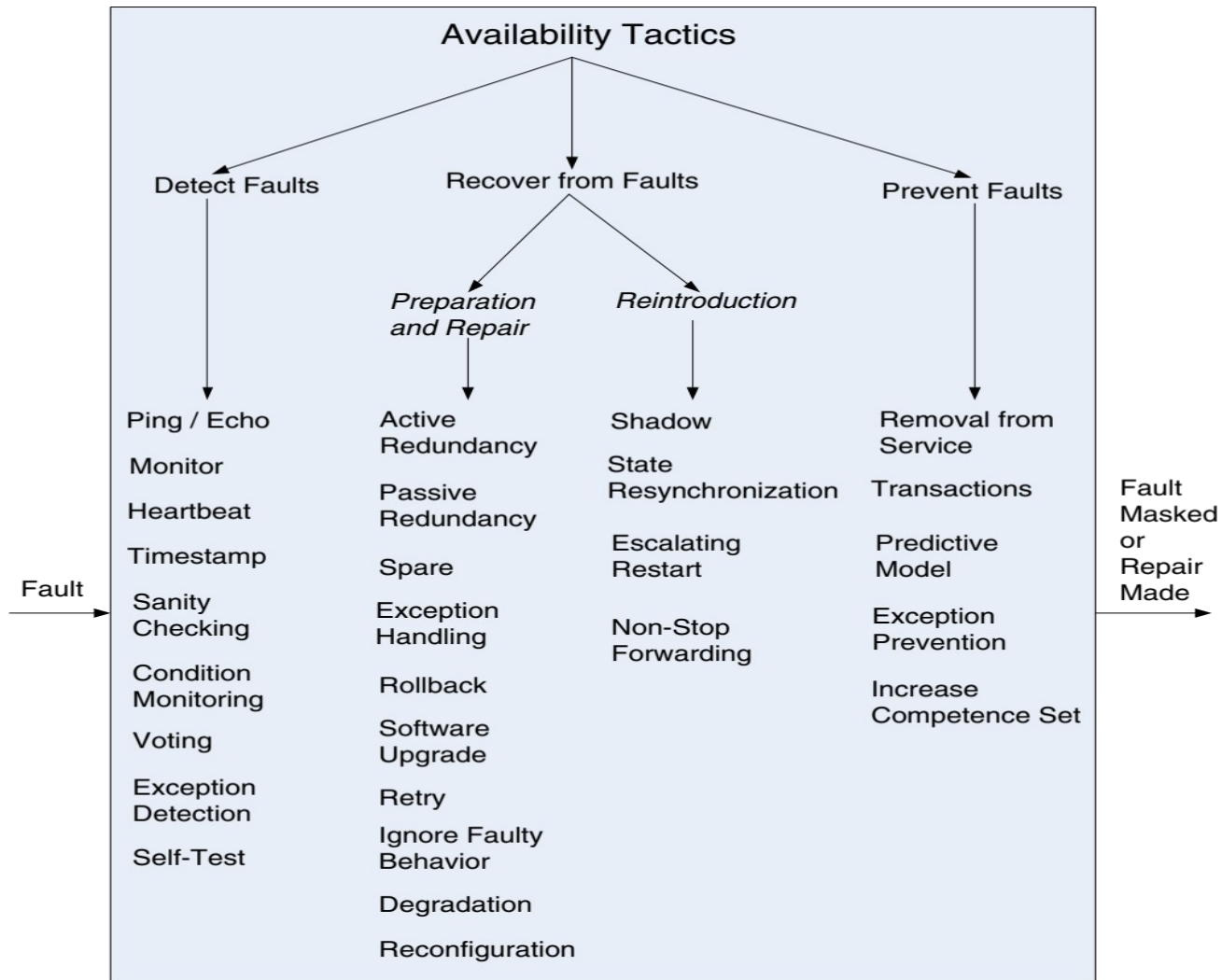
Goal of Availability Tactics

- A failure occurs when the system no longer delivers a service consistent with its specification.
 - » this failure is observable by the system's actors.
- A fault (or combination of faults) has the potential to cause a failure.
- Availability tactics enable a system to endure faults so that services remain compliant with their specifications.
- The tactics keep faults from becoming failures or at least bound the effects of the fault and make repair possible.

Goal of Availability Tactics



Availability Tactics



Interoperability

- **Interoperability** is about *the degree to which two or more systems can usefully exchange meaningful information*.
- Like all quality attributes, interoperability is not a yes-or-no proposition but has shades of meaning.

Interoperability General Scenario

Portion of Scenario	Possible Values
Source	A system
Stimulus	A request to exchange information among system(s).
Artifact	The systems that wish to interoperate
Environment	System(s) wishing to interoperate are discovered at run time or known prior to run time.
Response	One or more of the following: •the request is (appropriately) rejected and appropriate entities (people or systems) are notified •the request is (appropriately) accepted and information is exchanged successfully •the request is logged by one or more of the involved systems
Response Measure	One or more of the following: •percentage of information exchanges correctly processed •percentage of information exchanges correctly rejected

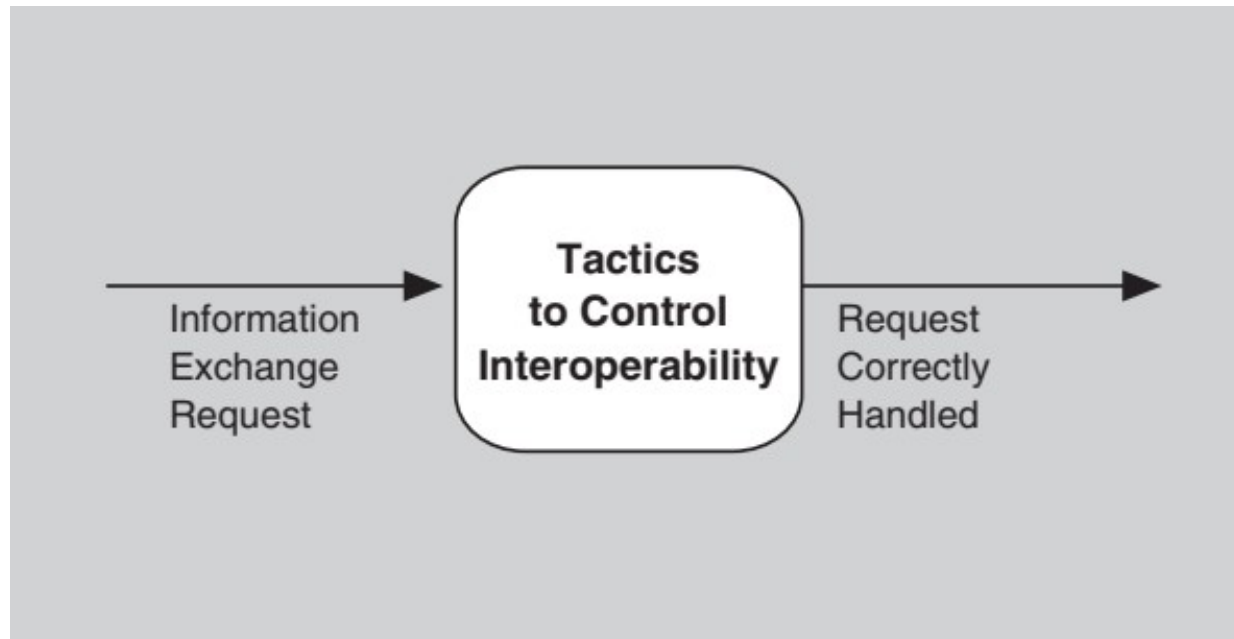
Sample Concrete Interoperability Scenario

- Our **vehicle information system** sends our current location to the traffic monitoring system.
- The **traffic monitoring system** combines our location with other information, overlays this information on a Google Map, and broadcasts it.
- Our location information is correctly included with a probability of 99.9%.

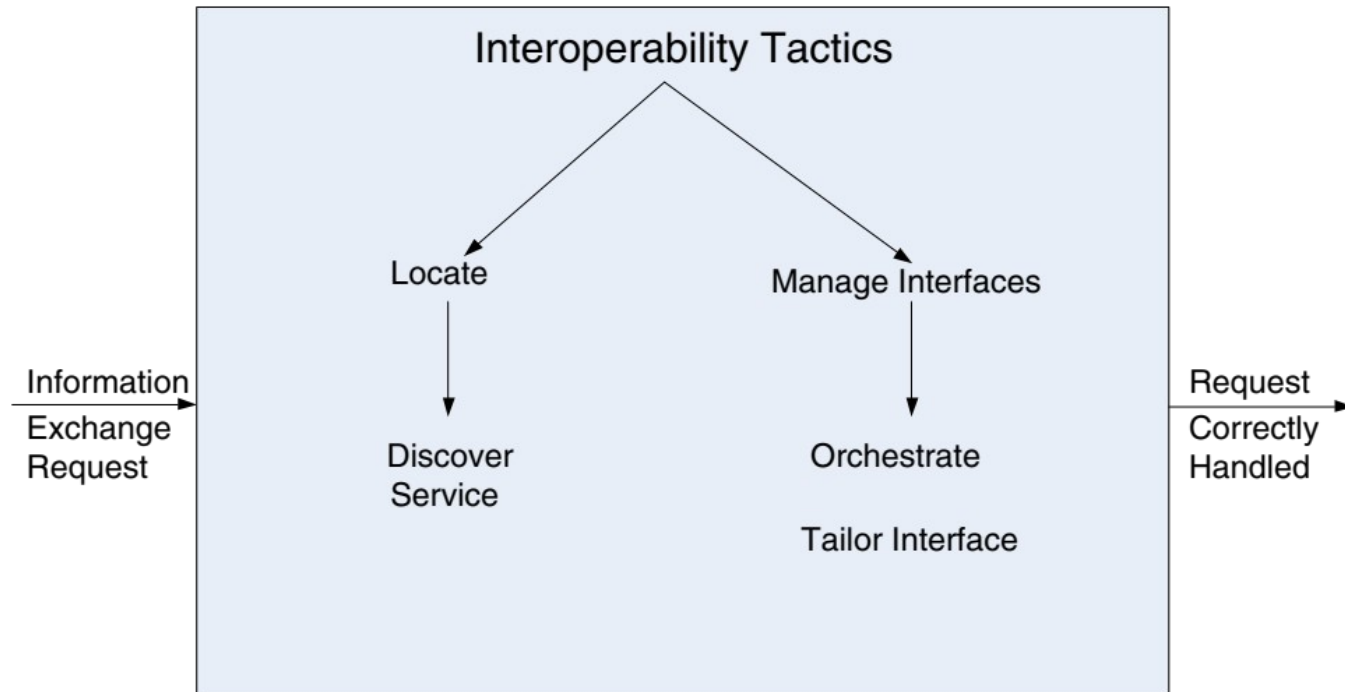
Goal of Interoperability Tactics

- For two or more systems to usefully exchange information they must
 - » Know about each other. That is the purpose behind the locate tactics.
 - » Exchange information in a semantically meaningful fashion. That is the purpose behind the manage interfaces tactics. Two aspects of the exchange are
 - Provide services in the correct sequence
 - Modify information produced by one actor to a form acceptable to the second actor.

Goal of Interoperability Tactics



Interoperability Tactics



Modifiability

- **Modifiability** is *about change and our interest in it* is in the cost and risk of making changes.
- To plan for modifiability, an architect has to consider three questions:
 - » What can change?
 - » What is the likelihood of the change?
 - » When is the change made and who makes it?

Modifiability General Scenario

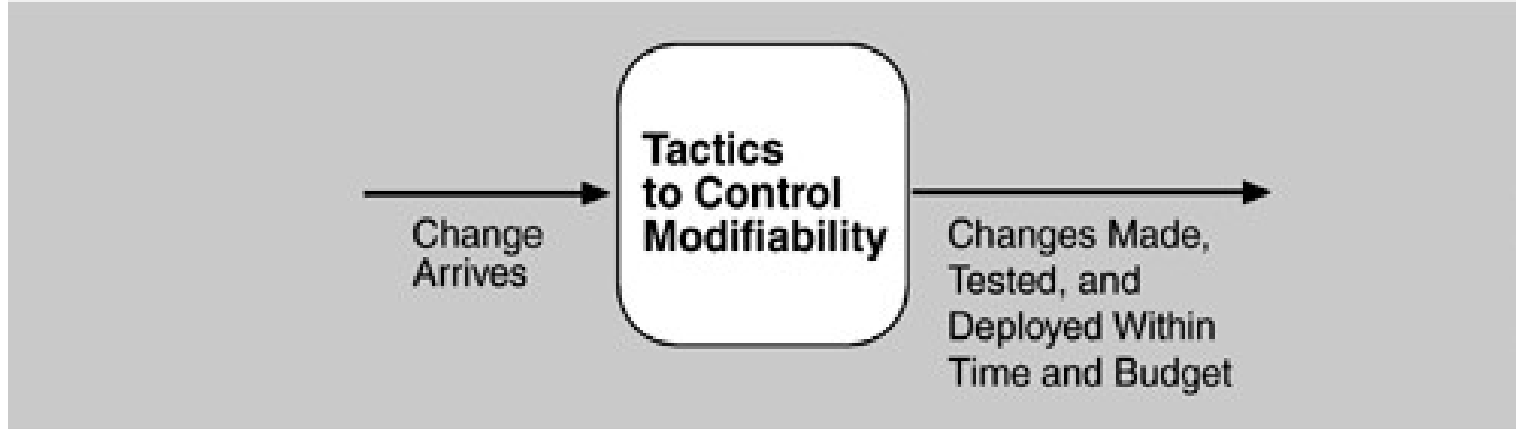
Portion of Scenario	Possible Values
Source	End user, developer, system administrator
Stimulus	A directive to add/delete/modify functionality, or change a quality attribute, capacity, or technology
Artifacts	Code, data, interfaces, components, resources, configurations, ...
Environment	Runtime, compile time, build time, initiation time, design time
Response	One or more of the following: •make modification •test modification •deploy modification
Response Measure	Cost in terms of: •number, size, complexity of affected artifacts •effort •calendar time •money (direct outlay or opportunity cost) •extent to which this modification affects other functions or quality attributes •new defects introduced

Sample Concrete Modifiability Scenario

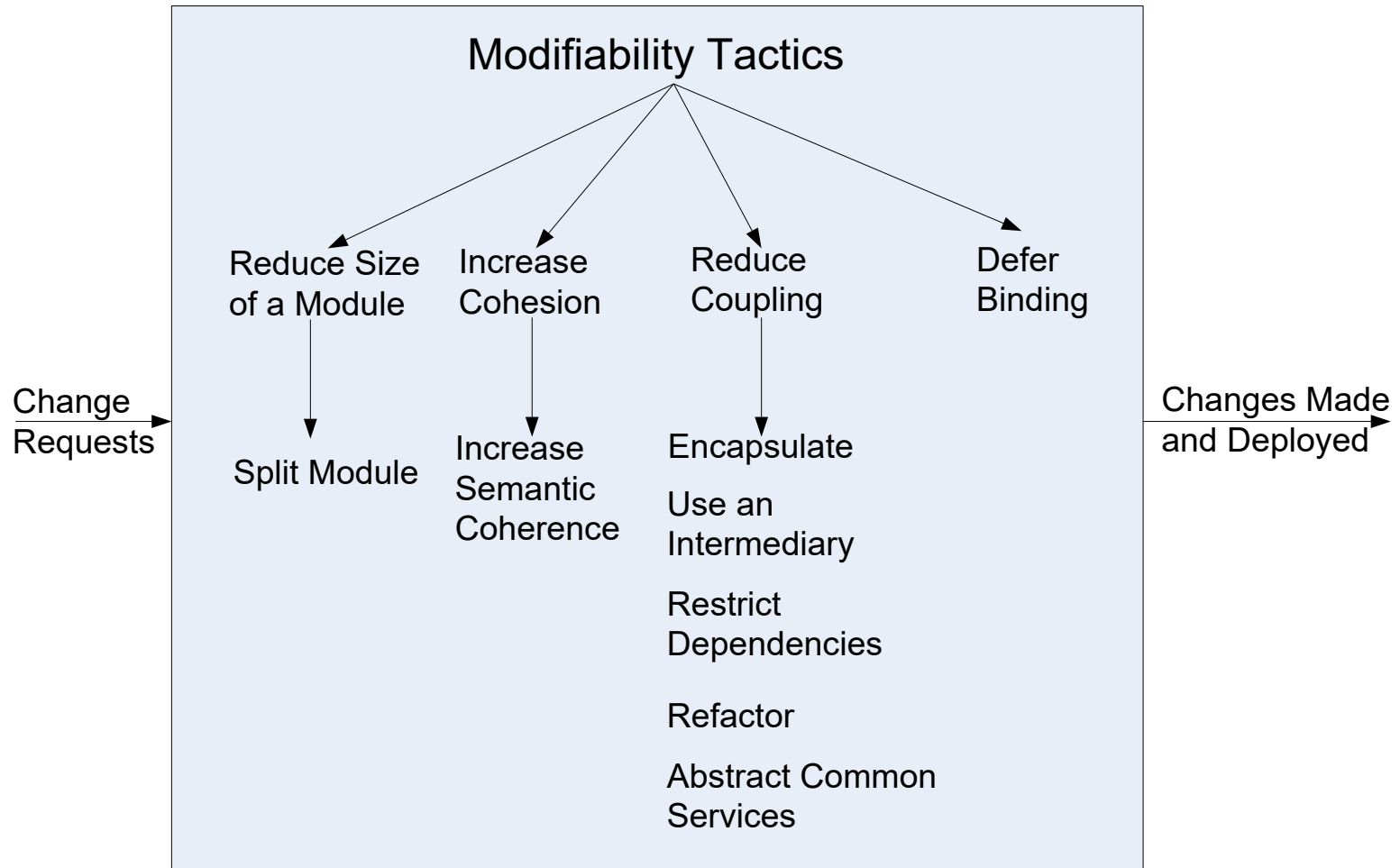
- The developer wishes to change the user interface by modifying the code at design time.
- The modifications are made with no side effects within three hours.

Goal of Modifiability Tactics

- **Tactics** to control modifiability have as their goal *controlling the complexity of making changes, as well as the time and cost to make changes.*



Modifiability Tactics



Performance

- **Performance** is about *time* and the software system's ability to meet timing requirements.
- When events occur – interrupts, messages, requests from users or other systems, or clock events marking the passage of time – the system, or some element of the system, must respond to them in time.
- Characterizing the events that can occur (and when they can occur) and the system or element's time-based response to those events is the essence of discussing performance.

Performance General Scenario

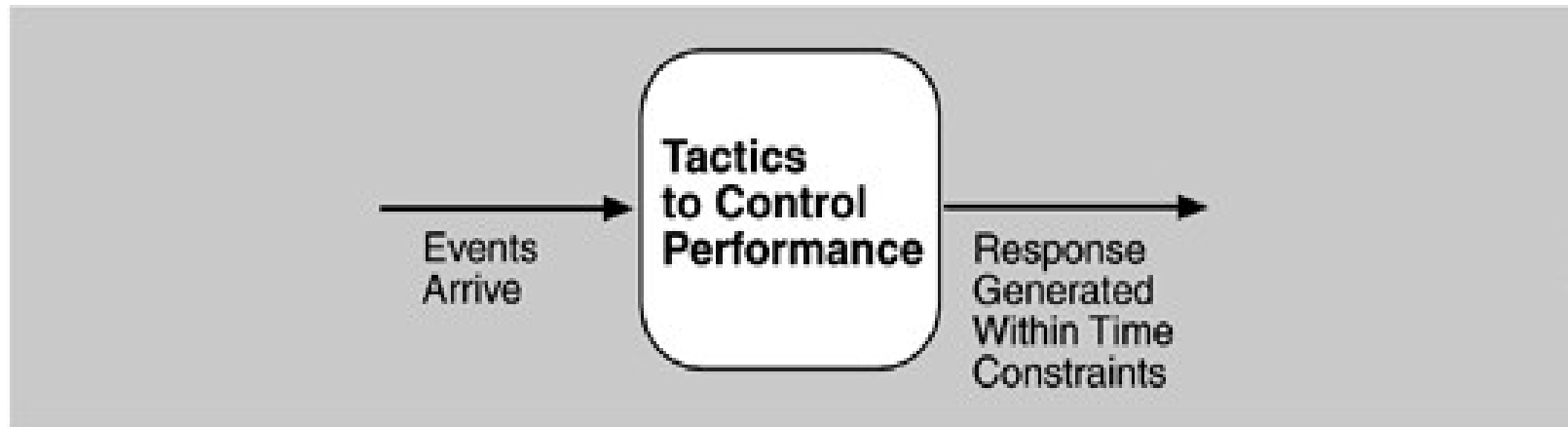
Portion of Scenario	Possible Values
Source	Internal or external to the system
Stimulus	Arrival of a periodic, sporadic, or stochastic event
Artifact	System or one or more components in the system.
Environment	Operational mode: normal, emergency, peak load, overload.
Response	Process events, change level of service
Response Measure	Latency, deadline, throughput, jitter, miss rate

Sample Concrete Performance Scenario

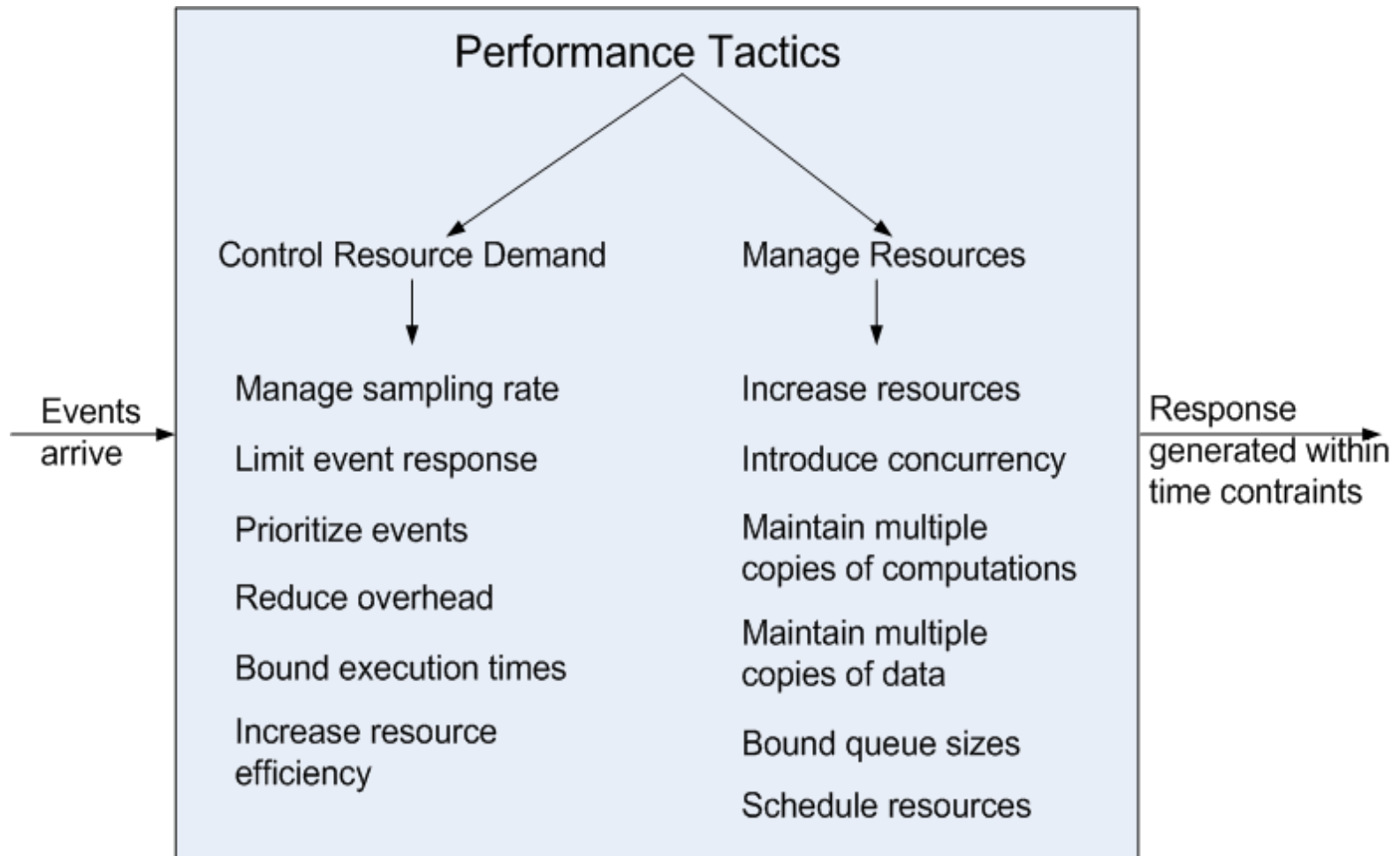
- Users initiate transactions under normal operations.
- The system processes the transactions with an average latency of two seconds.

Goal of Performance Tactics

- Tactics to control Performance have as their goal *to generate a response to an event arriving at the system within some time-based constraint.*



Performance Tactics



Security

- **Security** is a measure of the *system's ability to protect data and information from unauthorized access* while still providing access to people and systems that are authorized.
- An action taken against a computer system with the intention of doing harm is called an *attack* and can take a number of forms.
- It may be an unauthorized attempt to access data or services or to modify data, or it may be intended to deny services to legitimate users.

Security...

- Security has **three** main characteristics, called CIA:
 - » **Confidentiality** is the property that data or services are protected from unauthorized access. For example, a hacker cannot access your income tax returns on a government computer.
 - » **Integrity** is the property that data or services are not subject to unauthorized manipulation. For example, your grade has not been changed since your instructor assigned it.
 - » **Availability** is the property that the system will be available for legitimate use. For example, a denial-of-service attack won't prevent you from ordering a book from an online bookstore.
- Other characteristics that support CIA are:
 - » **Authentication** verifies the identities of the parties to a transaction and checks if they are truly who they claim to be. For example, when you get an e-mail purporting to come from a bank, authentication guarantees that it actually comes from the bank.
 - » **Nonrepudiation** guarantees that the sender of a message cannot later deny having sent the message and that the recipient cannot deny having received the message. For example, you cannot deny ordering something from the Internet, or the merchant cannot disclaim getting your order.
 - » **Authorization** grants a user the privileges to perform a task. For example, an online banking system authorizes a legitimate user to access his account.

Security General Scenario

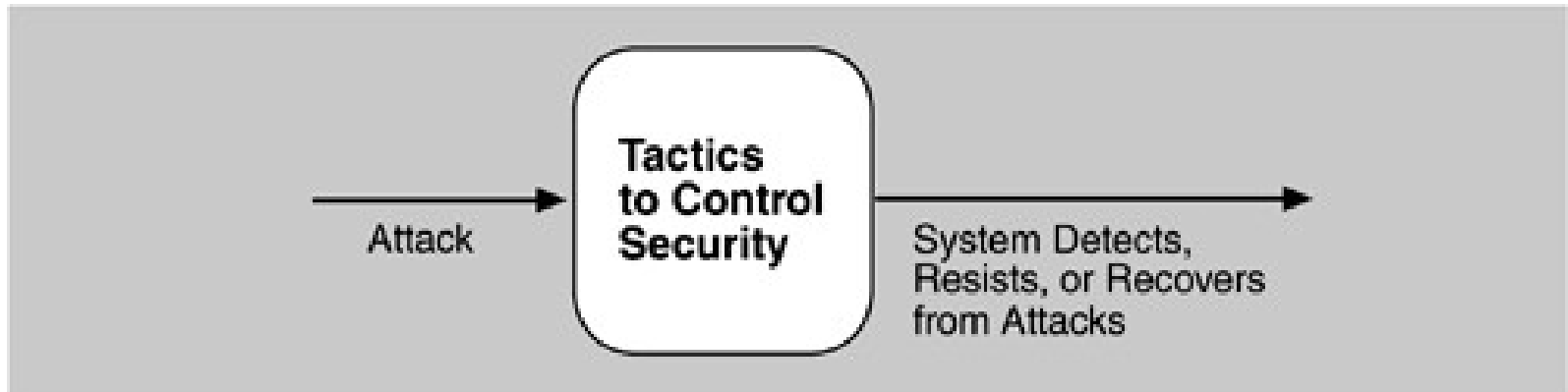
Portion of Scenario	Possible Values
Source	Human or another system which may have been previously identified (either correctly or incorrectly) or may be currently unknown. A human attacker may be from outside the organization or from inside the organization.
Stimulus	Unauthorized attempt is made to display data, change or delete data, access system services, change the system's behavior, or reduce availability.
Artifact	System services; data within the system; a component or resources of the system; data produced or consumed by the system
Environment	The system is either online or offline, connected to or disconnected from a network, behind a firewall or open to a network, fully operational, partially operational, or not operational
Response	Transactions are carried out in a fashion such that •data or services are protected from unauthorized access; •data or services are not being manipulated without authorization; •parties to a transaction are identified with assurance; •the parties to the transaction cannot repudiate their involvements; •the data, resources, and system services will be available for legitimate use. The system tracks activities within it by •recording access or modification, •recording attempts to access data, resources or services, •notifying appropriate entities (people or systems) when an apparent attack is occurring.
Response Measure	One or more of the following •how much of a system is compromised when a particular component or data value is compromised, •how much time passed before an attack was detected, •how many attacks were resisted, •how long does it take to recover from a successful attack, •how much data is vulnerable to a particular attack,

Sample Concrete Security Scenario

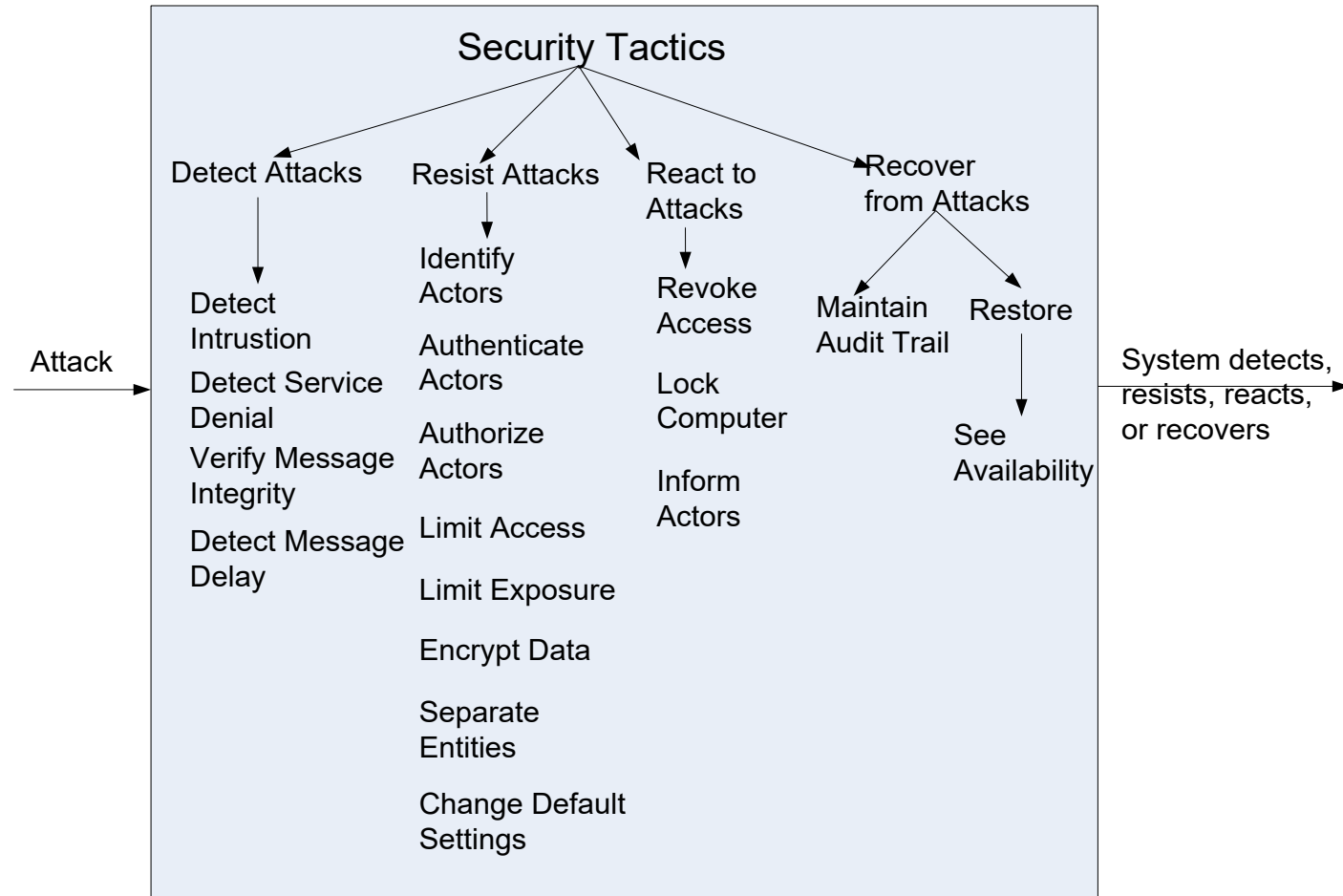
- A disgruntled employee from a remote location attempts to modify the pay rate table during normal operations.
- The system maintains an audit trail and the correct data is restored within a day.

Goal of Security Tactics

- One method for thinking about system security is to think about physical security.
- Secure installations have limited access to them (e.g., by using security checkpoints), have means of detecting intruders (e.g., by requiring legitimate visitors to wear badges), have deterrence mechanisms such as armed guards, have reaction mechanisms such as automatic locking of doors and have recovery mechanisms such as off-site back up.
- This leads to our four categories of tactics: **detect**, **resist**, **react**, and **recover**.



Security Tactics



Testability

- **Software testability** refers to *the ease with which software can be made to demonstrate its faults* through (typically execution-based) testing.
- Specifically, testability refers to the probability, assuming that the software has at least one fault, that it will fail on its *next* test execution.
- If a fault is present in a system, then we want it to fail during testing as quickly as possible.

Testability...

- For a system to be properly testable, it must be possible to *control* each component's inputs (and possibly manipulate its internal state) and then to *observe* its outputs (and possibly its internal state).

Testability General Scenario

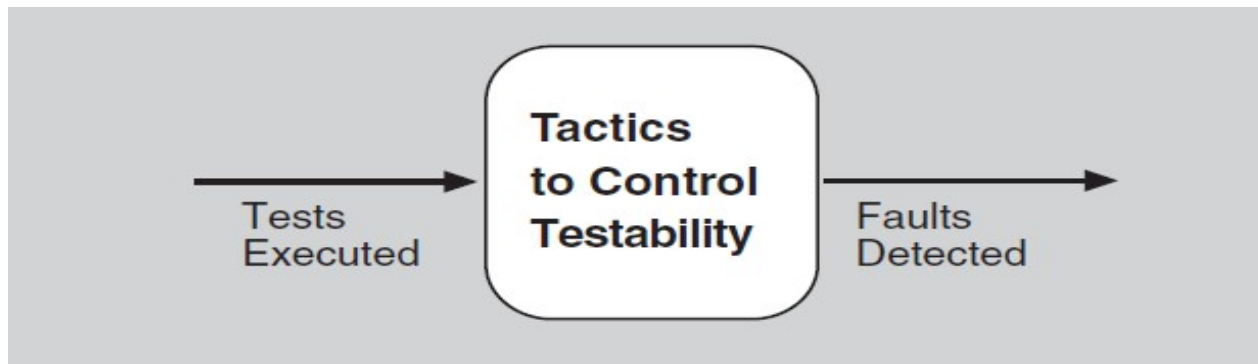
Portion of Scenario	Possible Values
Source	Unit testers, integration testers, system testers, acceptance testers, end users, either running tests manually or using automated testing tools
Stimulus	A set of tests are executed due to the completion of a coding increment such as a class, layer or service; the completed integration of a subsystem; the complete implementation of the system; or the delivery of the system to the customer.
Environment	Design time, development time, compile time, integration time, deployment time, run time
Artifacts	The portion of the system being tested
Response	One or more of the following: execute test suite and capture results; capture activity that resulted in the fault; control and monitor the state of the system
Response Measure	One or more of the following: effort to find a fault or class of faults, effort to achieve a given percentage of state space coverage; probability of fault being revealed by the next test; time to perform tests; effort to detect faults; length of longest dependency chain in test; length of time to prepare test environment; reduction in risk exposure ($\text{size}(\text{loss}) * \text{prob}(\text{loss})$)

Sample Concrete Testability Scenario

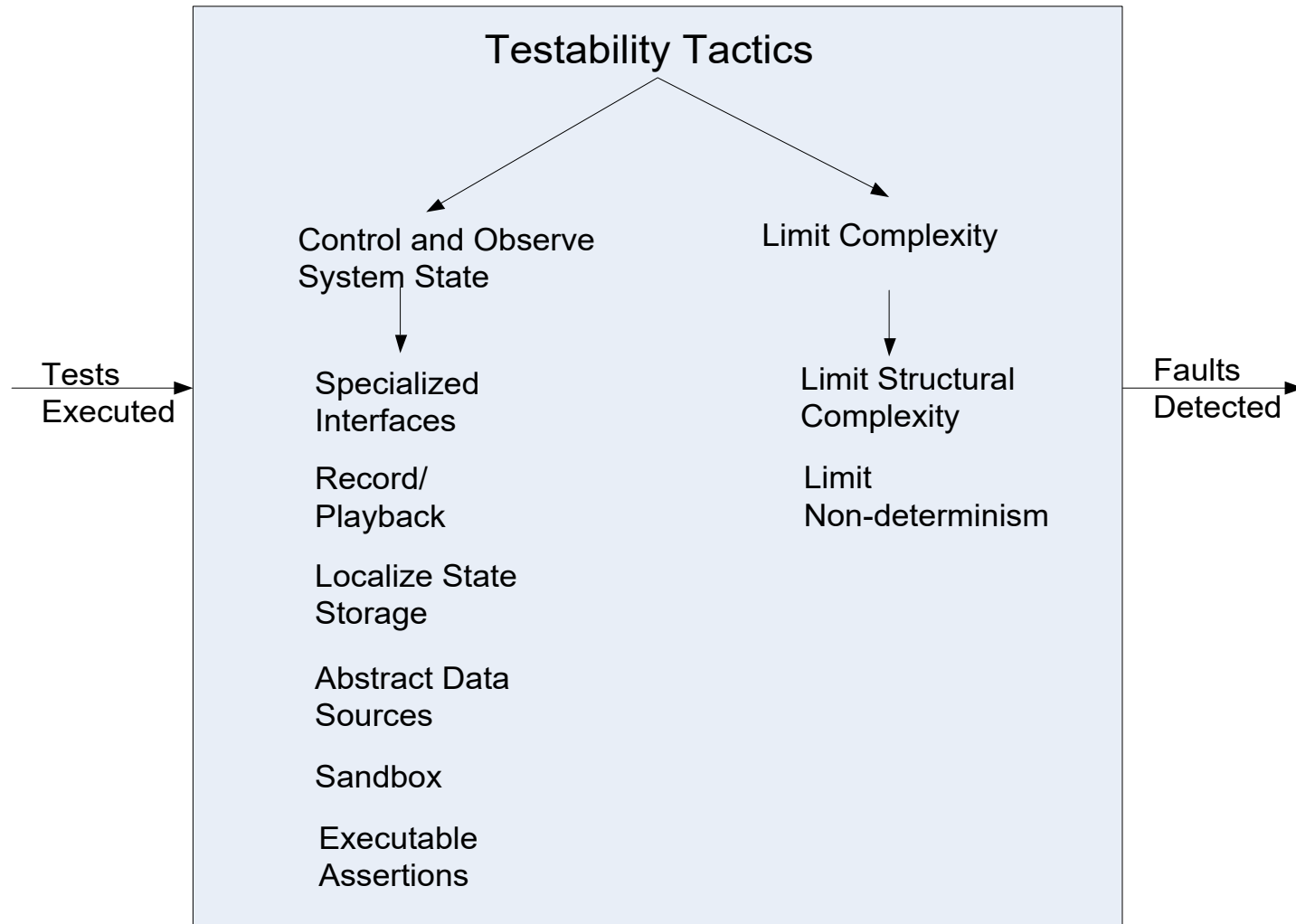
- The unit tester completes a code unit during development and performs a test sequence whose results are captured and that gives 85% path coverage within 3 hours of testing.

Goal of Testability Tactics

- The goal of tactics for testability is *to allow for easier testing when an increment of software development has completed.*
- Anything the architect can do *to reduce the high cost of testing* will yield a significant benefit.
- There are two categories of tactics for testability:
 - » The first category deals with adding *controllability and observability* to the system.
 - » The second deals with *limiting complexity* in the system's design.



Testability Tactics



Usability

- **Usability** is concerned with *how easy it is for the user* to accomplish a desired task and the kind of user support the system provides.
- Over the years, a focus on usability has shown itself to be one of the cheapest and easiest ways to improve a system's quality (or, more precisely, the user's *perception* of quality).

Usability...

- Usability comprises the following areas:
 - » Learning system features.
 - » Using a system efficiently.
 - » Minimizing the impact of errors.
 - » Adapting the system to user needs.
 - » Increasing confidence and satisfaction.

Usability General Scenario

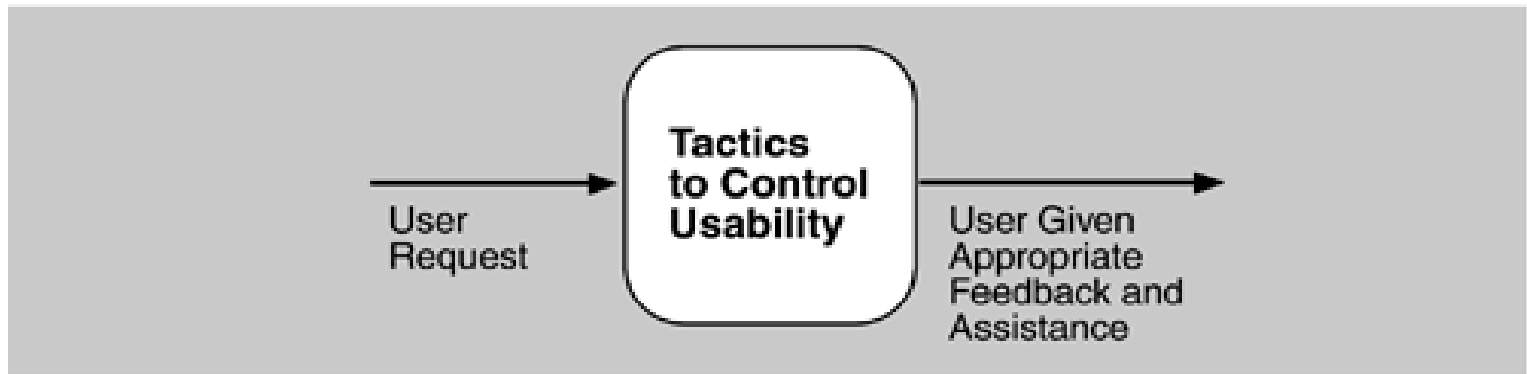
Portion of Scenario	Possible Values
Source	End user, possibly in a specialized role
Stimulus	End user tries to use a system efficiently, learn to use the system, minimize the impact of errors, adapt the system, or configure the system
Environment	Runtime or configuration time
Artifacts	System or the specific portion of the system with which the user is interacting.
Response	The system should either provide the user with the features needed or anticipate the user's needs.
Response Measure	One or more of the following: task time, number of errors, number of tasks accomplished, user satisfaction, gain of user knowledge, ratio of successful operations to total operations, or amount of time or data lost when an error occurs.

Sample Concrete Usability Scenario

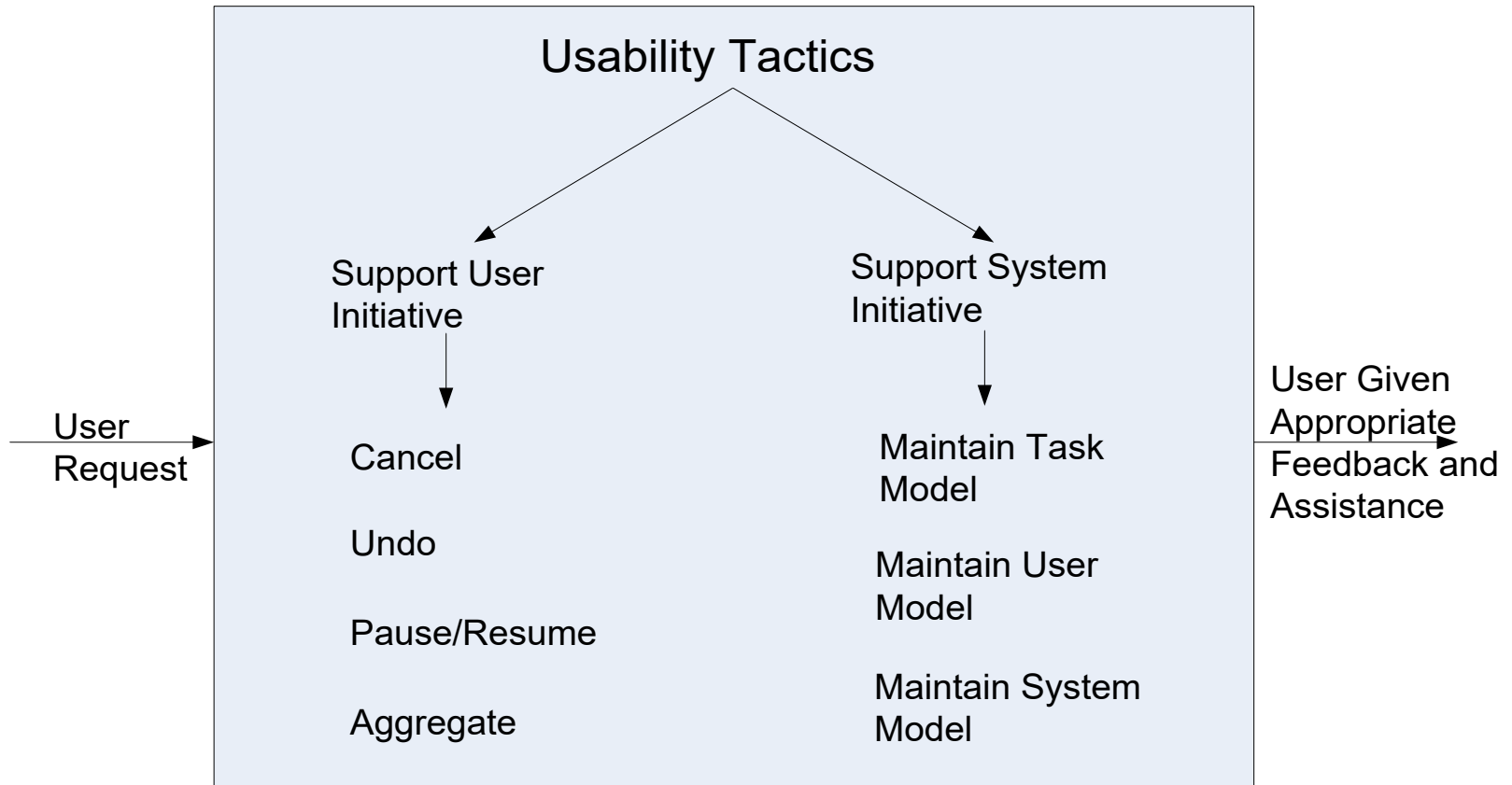
- The user downloads a new application and is using it productively after two minutes of experimentation.

Goal of Usability Tactics

- Researchers in human-computer interaction have used the terms "*user initiative*," "*system initiative*," and "*mixed initiative*" to describe which of the human-computer pair takes the initiative in performing certain actions and how the interaction proceeds.
- Usability scenarios can combine initiatives from both perspectives.
- We use this distinction between user and system initiative to discuss the tactics that the architect uses to achieve the various scenarios.



Usability Tactics



Reading (Self Study)

- Other quality attributes (Chapter 12 on the textbook)
- Quality attribute modelling and analysis (Chapter 14 on the textbook)